

RESEARCH

Open Access



Functionality-aware offloading technique for scheduling containerized edge applications in IoT edge computing

Lionel Nkenyereye^{1†}, Boon Giin Lee^{2†} and Wan-Young Chung^{3*†}

Abstract

Edge computing (EC) represents a basic functionality to support the efficiency of the future Internet of intelligent things. The EC has promoted container adoption for deploying and managing applications. Current container scheduling techniques on edge infrastructures show multiple limitations. The design scheduler of container applications execute workflow as specified by the container cloud workflow engine assisted by the Kubernetes platform. We model dependency workflow of containerized applications built using microservices as directed acyclic graph (DAG). The structure of DAG allows the system to prepare the scheduling order list of microservices. The Argo workflow is used to prepare the sequence to deploy containerized applications. In addition, edge worker nodes' resource utilization data enabled assists to select on which edge worker nodes the scheduling will take place. By combining the two mechanism, we termed the scheduling as functionality-aware offloading on scheduling containerized edge applications. We implemented the orchestration prototype and evaluate the performance of the proposed technique under extensive simulations using the ContainerCloudSim simulator with a module that models a lightweight Kubernetes platform in the context of the edge computing infrastructure. To validate our containerized edge inference service and collect data for the simulation setup, we used Raspberry Pis 4, and the cloud core was set up on Amazon Web Services. The workload in the pre-defined workflow using Argo K8s native was performed by calling the pre-trained model (downloaded and stored locally) and then executing the prediction microservice running on Raspberry Pis. The results demonstrate that our proposal outperforms the baseline scheduling offloading technique in edge computing by decreasing the average scheduling time of containerized edge applications by 15%.

Keywords Edge computing, Scheduling based, Container workflow engine, Containerized applications, Docker container, Directed acyclic graph, Kubernetes, ContainerCloudSim

Introduction

Internet-of-Things (IoT) devices, mobile computing devices (e.g., smartphones), and wearable body area network (WBAN) devices generate a huge amount of data [1]. Worldwide Global DataSphere IoT Device and Data Forecast estimated that in 2025, 42 billion devices will generate 79.4 zettabytes of data [2, 3]. This has accelerated the development of data processing and big data frameworks hosted on cloud data center to provide value-added services. There are several challenges (e.g., monetary cost, transmission delay, and privacy leakage) involved in moving and processing a large amount of data

[†]Lionel Nkenyereye, Boon Giin Lee and Wan-Young Chung contributed equally to this work.

*Correspondence:
Wan-Young Chung
chung.wanyoung@gmail.com

¹ AI Convergence Education & Research Group, Pukyong National University, Yongso-ro 45, Nam-gu 48513, Busan, Korea

² Nottingham Ningbo China Beacons of Excellence Research and Innovation Institute; School of Computer Science, University of Nottingham Ningbo China, Yinzhou, 315104 Ningbo, Zhejiang, China

³ Department of AI Convergence & Department of Electronic Engineering, Pukyong National University, Yongso-ro 45, 48513 Nam-gu, Busan, Korea

in the cloud data center. Edge Computing (EC) has been proposed to address the abovementioned challenges. EC moves cloud services to the edge of the network. Edge nodes are closer to IoT devices and data sources [4]. The cloud layer supports data storage, source code management, continuous delivery (CD), and continuous integration (CI). For fast deployment and migration at edge, applications are based on microservices and deployed inside containers. This type of application is called cloud-native application (CNA) [5].

Edge layer performed on-site data processing through containerized applications by processing and predicting new unseen data. Virtualization technology (e.g., virtual machine (VM), Docker container) has been proposed [6, 7] to enable the scheduling of CNAs. The implementation of CNAs might include several microservices. The job execution of these microservices is configured through workflow systems, such as Pegasus [8], BioDepot [9], Argo workflow [10].

Cloud to edge scheduling [7] is a technique in which a server and its workload are moved onto the edge of the network. The scheduling of workflow system on distributed edge nodes requires orchestrator that has both flexible and expandable features. For instance, Kubernetes (K8s) [11] is an open-source platform designed for provisioning CAs and their services. K8s schedules CAs through Docker runtime. The deployment is based on workflow system designed by Argo workflow. The deployment based on Argo workflow arrange tasks in a Directed Acyclic Graphs (DAG) to manage the dependencies and conditional execution and ordering tasks in the workflow. In addition, scheduling workflow algorithms hand off the task execution to K8s by optimizing the execution order of tasks and mapping them to computational resources [12]. K8s, through its core component Kube-scheduler [13], conducts pod scheduling. Kube-scheduler decides worker nodes that orchestrate newly created pods (or tasks in the context of Argo workflow). Although it bases its performance on a range of variables and algorithms, it has several restrictions and challenges that may prevent it from operating efficiently. For instance, the scheduling process may create a bottleneck in large clusters with thousands of nodes and pods due to the increasing complexity and number of decisions made. The Cluster of worker nodes could experience high interruption rates. In addition creation and deletion of pods causes the scheduler to select inappropriate nodes, which can affect negatively their placements [14, 15]. However, when running the workflow system to the K8s platform, the task pod scheduling of K8s does not adhere to the optimized task execution order determined by workflow scheduling algorithms. This discrepancy arises because Kube-scheduler prioritizes cluster load balance and

diverse CAs pod requirements [13], neglecting the precedence and dependency relationships inherent from workflow definition of microservices for the CAs encapsulated into pods [16]. Therefore, the scheduling of CAs to a container-based environment should identify the dependencies sequence of CAs prior to their deployment.

In the face of ongoing workflow demands and sudden spikes in resource requests, the cluster management node faces issue of resource availability, node affinities and tolerations. The dynamic nature of these requirements poses a significant resource allocation challenge for Argo workflow system within K8s in terms of resource allocation, then leading to unreliable tasks (or pods) placement and orchestration. Opting to monitor allocation of computational resources is a strategy that can manage peak loads in resource-intensive of CAs. However, this approach comes with drawbacks such as high costs and resource over-provisioning as the allocated resources remain underutilized during periods of low demand. It would be more convenient to monitor resource utilization to ensure sufficient computation resources to apt requirements of workflow engine that implement the placement of CAs, then provide relevant edge services.

Presently, the scheduler mechanism of CAs implemented with different scheduler algorithms and techniques is evolving in IoT edge computing [17] and [18]. The work in [19] uses serverless workflow engine to implement microservices. The big data based workflow in [18] was used to execute workflow steps by leveraging long-lived containers and handling interaction with various data sources via containers. As the research on container orchestrator won the attention of cloud native applications, cluster scheduler of long running CAs on K8s in the cloud-native era, Medea [20] has own over default kube-scheduler and Metis [21] with technology advantages, introducing formal semantics to capture interactions among with formal and across applications. Before Medea, many researchers have attempted to optimize the deployment of CAs in edge computing for large-scale cloud-edge containerized services.

To address the two above challenges, we propose a scheduling technique for the deployment of CAs on edge computing by combining the workflow engine to force the dependency and the efficiency of resource utilization on edge nodes. We assume that the execution of tasks in the workflow pipeline involves dependencies of software components or waits for some events to occur. Therefore, the proposed scheduling would reduce the scheduling latency [5].

To this end, the microservice scheduler system using containers' resource consumption and their workflow steps would facilitate the scheduling of CAs on heterogeneous edge nodes. In this paper, we aim to minimize

the orchestration time of CAs. To achieve that, the deployment dependency is modeled as a DAG. The order list computed from DAG constitutes microservices instances. The scheduling of CAs in this paper has two phases: pre-scheduling and computing edge nodes' resource consumption. In the pre-scheduling phase, an algorithm defines container-native workflow steps by composing them in a pre-defined list while orchestrating on-edge lightweight K8s worker instances known as pods. Next, the proposed scheduling controller uses the worker nodes' resource consumption algorithm to compute the input decision on which K8S nodes to instantiate CAs.

The main contributions of this study are summarized as follows:

- We formulate the container provisioning workflow steps model in edge computing. Considering the nature of CAs, we propose a functionality-aware offloading technique for scheduling CAs, termed FAOFE. The FAOFE aims at orchestrating CAs that expose edge services on top of distributed edge nodes.
- We focus on a container scheduling technique and propose an algorithm that use Argo workflow engine to encapsulate the execution sequence of dependencies attached to CAs. The proposed algorithm is captured in first step of the proposed scheduling technique. In the second stage of the proposed scheduling technique, the edge nodes' resource availability and orchestration decision are integrated into an offloading controller to allow the scheduling and orchestration of CAs.
- We implemented the orchestration prototype and evaluate the performance of FAOFE under extensive simulations using the ContainerCloudSim simulator within an infrastructure of K8s cluster.

The rest of the paper is organized as follows. [Related work](#) section presents the related work. [System design](#) section presents the system design. [Functionality-aware offloading Technique for scheduling containerized edge Application](#) section presents the functionality-aware offloading technique for scheduling CAs. [Evaluation](#) section evaluates the performance of the proposed technique. Finally, we present conclusions and future work in [Conclusion](#) section.

Related work

The scheduling approach for container-based applications is among the most important features used in EC for enabling advanced IoT applications. In this section, we review the development of scheduling strategy to CAs

and discuss three categories of scheduling strategy from the perspective of various parameter decisions namely constraints based scheduling, deep learning-based solutions, and K8s based cloud native workflow engine. These parameters supported the scheduling of CAs deployed as microservices. The scheduling orchestration for container-based microservices basically answers the “where to schedule” question, which has been examined for IoT applications.

Constraints based scheduling systems

Hu et al. [22] propose CEC, a containerized edge computing framework. CEC integrates workload prediction and resource pre-provisioning to enable low latency of user service requests and high utilization of edge resources. It is a self-adaptive controller to tune the resource for containers. Serverless EC execution through container scheduling was studied in [19]. This approach considered the extreme workload, device heterogeneity, and geographical dispersion of EC and storage infrastructure.

Cluster schedulers for CAs at the edge are continuously evolving to meet diverse requirements, encompassing workload prediction, network bandwidth, resource availability, and the heterogeneity of edge nodes. These constraint-based systems predominantly leverage K8s as the container orchestrator of choice. Investigations into network, latency, traffic between container nodes, and real-time information about edge nodes are underway to optimize the scheduling of cloud-native applications on K8s. Geographical constraints are crucial in implementing distributed services for EC. Santos et al. [17] proposed a network-aware scheduling approach for container-based applications validated on the K8s platform. This approach demonstrated an impressive 80% reduction in network latency for smart city container-based applications. The authors [23] specifically addressed latency issues arising from fetching container dependencies during the deployment of containers handling tasks at the edge. They implemented a dependency scheduling approach in K8s to orchestrate containers efficiently. Kim et al. [24] introduced a “science goal” as a mechanism for users to define scientific objectives and conditions of interest. To abstractly run applications on a collection of edge computing nodes, a two-layered scheduler comprising a cloud scheduler and an edge scheduler was implemented.

Chima et al. [25] introduced an enhanced design that incorporates real-time information about edge devices into the scheduler decision algorithm. This integration enables advanced orchestration and management, along with improved fault tolerance capabilities at the network edge. The preference is to account for physical, operational, and network parameters, in addition to

software states for dynamic application orchestration. Kim et al. [26] suggested a local scheduling scheme that manages user traffic locally at each edge node. The results of the performance evaluation demonstrated that local scheduling surpasses the existing load-balancing algorithm in the edge computing environment. This superiority is observed in terms of computational resource distribution and delay between edge nodes. Dias et al. [27] introduced a novel scheduling strategy named Infrastructure Aware, which considers factors such as network bandwidth availability and the download queue on each node. This strategy facilitates the fast and continuous provision of applications on geographically distributed heterogeneous devices at the edge, often with constrained resources. Joao et al. [28] proposed Geolocate, a generic scheduler designed for workload orchestration and distribution in containerized edge deployments.

Taking a different approach, efforts have been made to optimize the scheduling mechanism on K8s for deploying containerized applications with a focus on resource provisioning to facilitate low-latency, semantically meaningful interactions among containers and address CPU and GPU workload constraints. In the context of serverless EC execution through container scheduling, an exploration was conducted, considering factors such as extreme workload, device heterogeneity, and geographical dispersion of EC and storage infrastructure. Garefalakis et al. [21] introduce Medea, a novel cluster scheduler designed for placing both long and short-running containers. Medea employs powerful placement constraints with formal semantics to capture interactions among containers within and across applications. For long-running containers, it utilizes an optimization-based approach considering constraints and global objectives. For short-running containers, it employs a traditional task-based scheduler to minimize placement latency. Jiang et al. [29] introduce CoEdge, a novel edge containerization method that allows an edge node to simultaneously handle deep learning (DL) tasks by segregating CPU and GPU workloads into distinct containers. CoEdge integrates a hierarchical DL task scheduling framework, distributing DL tasks to edge nodes based on their computational profiles, communication overhead, and real-time demands. Kum et al. [30] present a method aimed at increasing GPU resource utilization by modifying the batch size of the input in an inference application. The proposed approach involves identifying inference pipelines to understand how different resources are used, measuring the effect of batch inference on GPUs. The method comprises modules for batch size management, capable of adjusting batch size according to available resources,

and model management, supporting on-the-fly updates of a model.

These schedulers for CAs are established on predefined objectives, operate by adhering to a set of predetermined design goals aligned with the distributed topology of virtualized infrastructures. These goals encompass considerations such as computational resource distribution, delay, network bandwidth, download queue, and the cached layer of containerized images. However, this predefined design relies on operators to manually articulate numerous constraints, aiming to tailor the scheduling mechanism for minimizing deployment latency and ensuring the availability of applications across extensive distributed edge computing zones. Despite the efforts to optimize deployment, these schedulers encounter limitations, particularly in the area of artificial intelligence solutions with a significant number of task dependencies. The scheduling of such tasks struggles to adhere to the specified constraints related to the placement of CAs. While these schedulers offer qualitative performance, they lack the ability to enhance overall efficiency through self-learning decisions derived from experiences. Therefore, the challenges arise from the intricate manual specification of placement constraints, where there is a constant effort to satisfy as many constraints as possible.

Deep learning-based container schedulers

Deep reinforcement learning (DRL) has been employed in K8s schedulers to enhance the performance of CAs, addressing the limitations of qualitative placement constraints. Wang et al. [20] introduce Metis, a versatile scheduler that utilizes DRL techniques to learn the optimal placement of CAs. This approach eliminates the need for complex manual specification of placement constraints, providing concrete and quantitative scheduling criteria. Metis incorporates a novel hierarchical learning technique that decomposes the intricate container placement problem into a hierarchy of subproblems, significantly reducing the state and action space to scale the RL agent. Tang et al. [31] propose a multiagent task scheduling algorithm based on the dueling double deep recurrent Q-network method. This algorithm is designed to approximate the optimal task scheduling and resource allocation solution. The task scheduling competition process is formulated as a partially observable stochastic game, allowing serverless edge computing nodes to non-cooperatively schedule tasks and allocate computing resources based on their locally observed system state. This approach takes into account the task generation state, data queue state, communication channel state, and previous computing resource allocation state.

The implementation of DRL scheduler for containerized applications minimizes the need for intricate manual

configurations before placing containerized applications. This results in an enhancement of the approximate schedule for subsequent task scheduling and resource allocation. The observation of resources on edge computing nodes introduces additional criteria for the DRL agent, allowing the learning process to scale whenever the conventional scheduler encounters difficulties. Nevertheless, the task generation state does not account for task dependencies within the workflow system, data transmission, and high concurrency among tasks. These omissions significantly compromise the performance of container-based workflow systems.

K8s based cloud native workflow engine

The trend of integrating workflow systems with K8s is gaining popularity, facilitated by the implementation of workflow containerized systems. These systems can later be incorporated into the development of scheduler policies for deployment and autoscaling of application services on K8s. DAGs can simulate interdependent data processing tasks in this context. For instance, Andrei-Alin et al. [18] proposed a big data workflow centered around software containers. Their solution took into account available data locality information, employed long-lived containers for executing workflow steps, and managed interactions with various data sources through containers. Another approach by Chenggang et al. [10] introduced a cloud-native workflow engine called KubeAdaptor, serving as a docking framework to enable workflow containerization on K8s. It integrates workflow systems with K8s, ensuring the consistency of task scheduling order. The architecture of KubeAdaptor along with detailed functionality implementation and the event-trigger mechanism was elaborated. In comparison to the baseline Argo workflow engine, KubeAdaptor demonstrated superior performance in terms of average execution time of task pods, average workflow lifecycle, and resource usage rate. Feng et al. [32] proposed a three-step scheduling model to combine the scheduling of container-based workflows with the deployment of containers in a cloud-edge environment. In the first step, virtual CPU (vCPU) is allocated for each container to facilitate vCPU sharing among different containers. Subsequently, a two-step resource deployment is employed to schedule the containers onto VMs.

None of the existing literature considered both container deployment workflow engine and containers' resource consumption in their proposed scheduler approach for the orchestration of CAs. Owing to the reasons mentioned above, we assume container deployment workflows by leveraging the DAG concept would scale the scheduling of CAs in form of microservices inside a K8s pod.

A qualitative comparison

This study centers on CAs in EC. The CAs are structured as interconnected microservices with a designated deployment workflow. The container deployment workflow engine manages these dependencies and executes them in accordance with the specified sequence within the K8s cluster. Additionally, our study implements CAs at edge through offloading. We use the term offloading and scheduling interchangeably. The proposed scheduling technique employs Argo workflow engine to capture the execution order of dependencies of CAs. It also constantly monitors the resource utilization on edge nodes so that the scheduling of workflows will take place on nodes with sufficient computation resources.

In comparison to existing research, the uniqueness of our framework is encapsulated in two key aspects. First, it introduces a functionality-aware approach, termed FAOFE, which incorporates the Argo workflow engine and edge resource management for scheduling CAs. This technique enhances the efficient placement of CAs on diverse edge nodes by addressing the inconsistent between workflow scheduling algorithms and K8s for workflow task scheduling order. FAOFE leverages workflow scheduling algorithms to optimize task execution order and maps tasks and resources, enhancing performance and execution efficiency. It implements a controller interfacing with existing container orchestration systems. Second, the FAOFE controller determines the orchestration of CAs on specific edge nodes through edge node resource management. This decision is facilitated by collecting cluster-wide metrics from all Kubelet daemons (K8s agents) on worker nodes. Kubelet gathers metrics such as CPU, disk space, and memory utilization. Over time, more knowledge accumulates the efficiency of scheduled containerized decisions for a specific edge node. This accumulated efficiency is leveraged to continually enhance the future orchestration of CAs. This capability allows FAOFE to dynamically offload and orchestrate CAs, minimizing scheduling and completion time while ensuring system stability.

The analysis of related work is summarized in Table 1. The infrastructure type can be a edge or a cloud edge, where a edge includes distributed worker nodes and heterogeneous computing systems, where container placement cost can be relevant (such as in [26]). Application types of interest are microservices of CAs, serverless, single workflows, or multiple workflows arranged in DAG concept. The edge node selection can be static, where the scheduling process can plan the tasks in advance, or dynamic, where the algorithm has to deal with unpredictable placement of pods deployed following a workflow system such as Argo workflow. The provisioning methods are divided into dynamic and static approaches,

Table 1 Summary table of related work on workflow engine and edge provisioning

Ref	Edge Node selection	Edge workload	infra	Placement	Edge orchestrator	Constraints	Deploy time	K8s	Workload engine	Metrics on Host and its containers
[27]	X	X	Edge	✓	X	X	X	X	X	X
[15]	X	X	Edge	✓		network aware	X	✓	X	X
[16]	X	X	Edge	X	✓	Startup Latency, resource efficient	X	✓	X	✓
[17]	X	X	Edge	✓	✓	X	X	X	X	X
[28]	X	X	Edge-cloud	X	✓	X	X	docker	X	X
[19]	✓	X	Edge	X	✓	data storage and registry proximity, compute resource	X	✓	serverless	✓
[29]	X	X	Edge-cloud	X	✓	kube-scheduler	X	✓	X	X
[30]	X	X	Edge-Fog	X	✓	resource discovery	X	✓	X	X
[22]	X	X	Edge	X	✓	geolocation	X	✓	X	X
[31]	X	X	Edge	✓	✓	resource constraints	X	✓	X	X
[32]	✓	✓	cloud-edge	✓	✓	X	✓	✓	Kube Adaptor	X
[24]	X	X	Cloud	X	X	Locality	X	✓	Bigdata	X
Our work	✓	✓	cloud-edge	✓	✓	Locality resource constraints	✓	✓	Argo	✓

where static means that the provisioning plan is prepared prior to execution. To simplify, we classify all the algorithms for resource constraint and deep learning systems into the static category, since these systems do not allow for creation of new resources on demand, even if some mechanisms for dynamic provisioning such as reinforcement learning agents [31] are available. The related work addresses multiple optimization objectives, including container placement time, reliability, throughput, resource or fairness, and there are examples of multi-criteria optimization among them. Most of the scheduling approaches also consider constraints for optimization, and the most commonly addressed ones are resource and location of worker nodes. Other constraints, such as workload prediction, dependency in workflow or container service throughput, are also present. Finally, we divide the algorithmic methods into workflow scheduling in K8s, including various list scheduling techniques based on big data workflow, KubeAdaptor that use Docker runtime and event-trigger algorithms, and the approaches using container scheduling on top of virtual machines.

An analysis suggests that there are some approaches to workflow scheduling and resource provisioning on clouds, but there is still room for improvement. New infrastructure types, including edge cloud and distributed edge should be investigated, more emphasis should be put on problems with multiple interrelated workflows such as Argo handed to K8s for provisioning, and new methods for CAs at edge with dynamic workloads and container's resource consumption and edge node selection algorithms offer potentially interesting areas of research. Finally, other objectives

and constraints in addition to placement of tasks (or pods) in Argo workflow and execution time can be explored.

In this context, we consider our attempt to address edge CAs workflow to orchestrate CAs using both Argo workflow inspired by DAG concept inside K8s cluster and dynamic edge node's resource consumption algorithms presented in this paper an interesting and original contribution that has not been explored before.

System design

Argo workflow for containerized applications

The CA is modeled as a complex multi-program application with the defined dependencies of workflows that can easily be represented as DAG [18]. We follow the Argo workflow known as a containerized native workflow engine. This workflow is natively integrated with K8s. It is compatible with K8s cluster through the customer resources definition (CRD). Each task runs in the pod. The workflow is based on DAG and other dependency topology. We model the K8s cluster as lightweight so that it will run on constraints resources edge devices.

The lightweight K8s platform such as K3s [33] is suitable on constrained machines and would be able also to support deployment workflow to process CAs and run them in the sequence specified by users. The illustrated components of running containers are shown in Fig. 1. This is a cloud-native workflow execution on a K8s cluster to support constrained resources machines as K8s edge node workers. The K8s controller has a configurable number of virtual worker machines that host each workflow running in the cluster. At submission time, each workflow (container) is

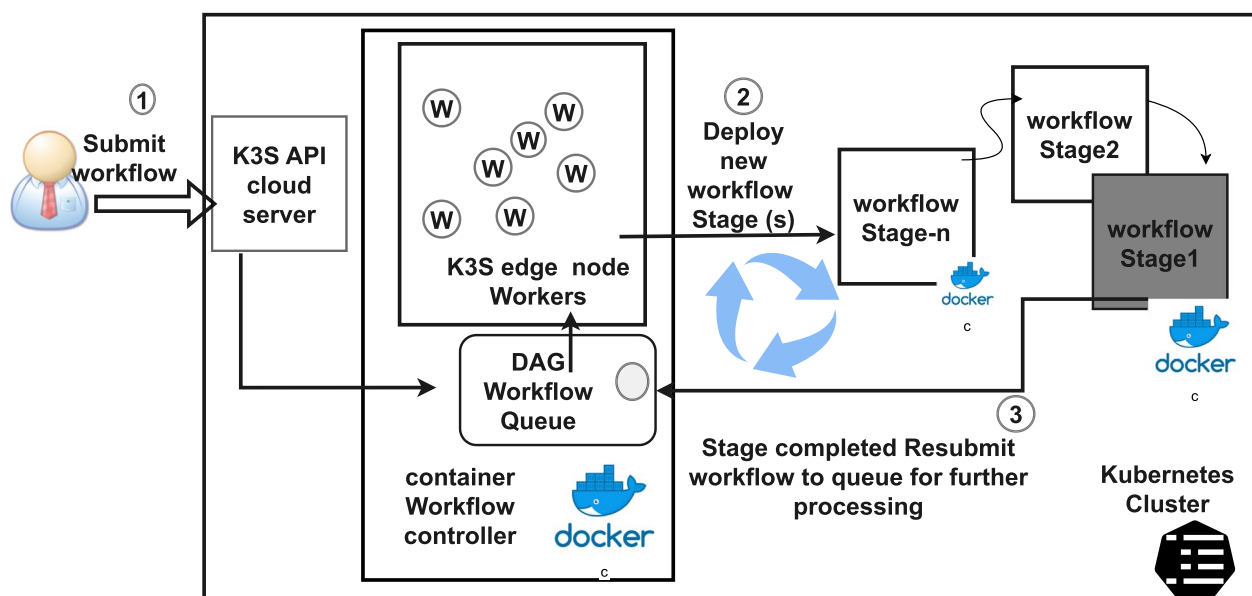


Fig. 1 Exemplary workflow for deployment of containerized applications using K8s

pushed into the workflow queue (step 1). One of the virtual worker machines will pick it up from the queue and initiate its execution. Each runnable stage is scheduled as a K8s pod assigned to a worker node (step 2 in Fig. 1). When a stage is completed, the pod event is triggered to insert the workflow in the workflow queue (step 3 in Fig. 1).

System model

We present the system model for scheduling containerized EA over edge computing nodes. The system model

is shown in Fig. 2. End users submit IoT sensor data to the edge device. Upon receiving the users' requests and realizing there is no CA instance, the service deployment interface submits the orchestration of CA to the K8s controller located at the cloud computing data center. The CA microservices are organized in deployment workflows as DAGs [34]. The K8s controller computes an orchestration scheme according to the DAG structure of workflow steps specified in K8s's template as well as the resource utilization of edge servers. The time series of edge servers'

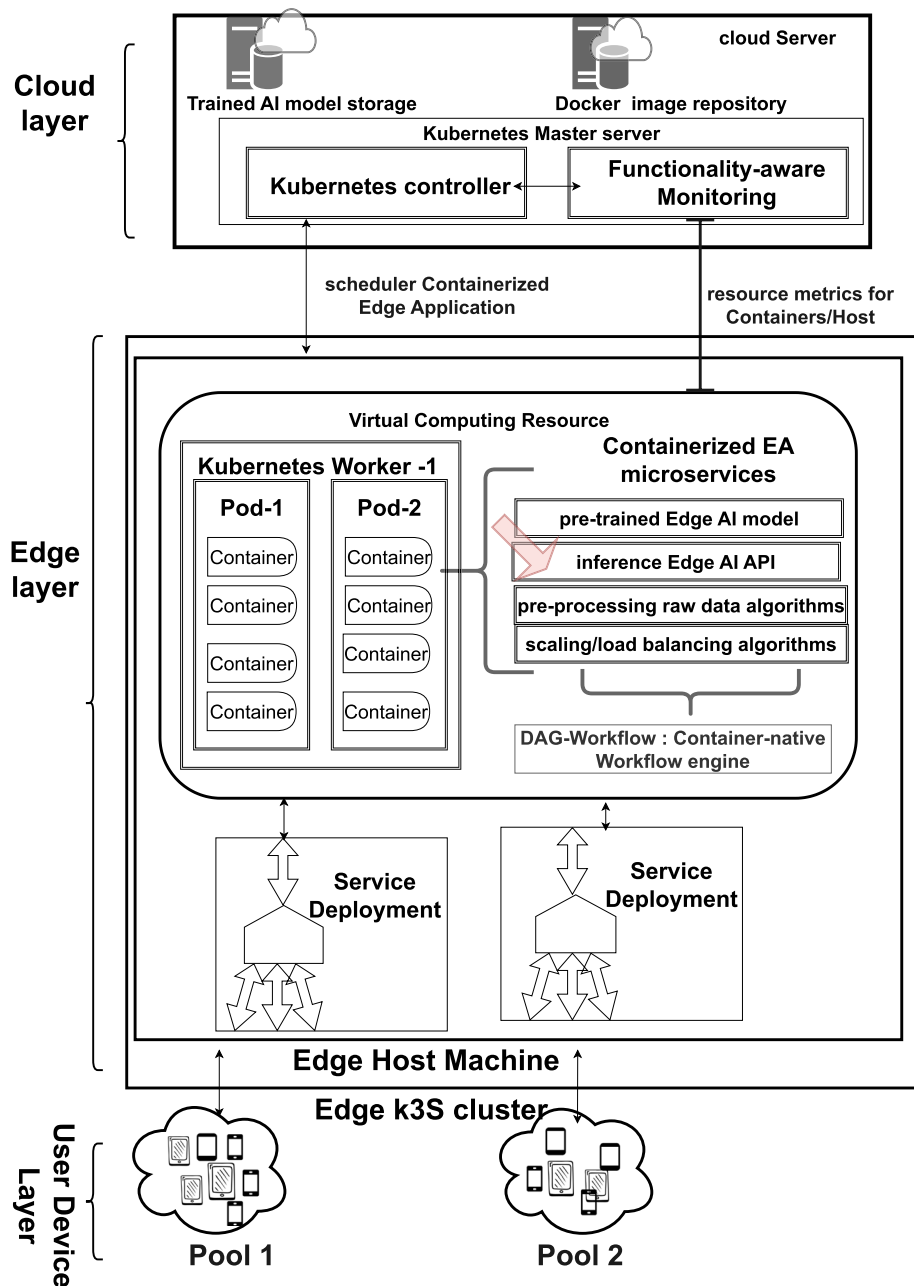


Fig. 2 System model for scheduling containerized applications-assisted with Argo workflow on K8s cluster for edge computing system

resource data is saved, then retrieved by the functionality-aware monitoring controller. Meanwhile, the K8s controller will refer to the functionality-aware outputs to estimate the completion of the orchestration of CAs on edge servers. The orchestration on the edge server will download and deploy necessary container images, execute the workflow steps, and return the resource consumption to the functionality-aware controller.

The framework includes multiple edge (i.e., K3s cluster) servers, cloud servers, and image registry (Docker repository) where the images of the CA are hosted. The edge servers are K8s workers that run the CAs inside the Pod and execute the workflows (i.e., microservices). In the system model, the edge nodes could not maintain the containers' instances running, even when there were no CA tasks to be executed. The availability of CAs is defined per user demand. The request for using the scheduling call triggers the workload orchestration of CAs. In this case, if the closest edge nodes near the user do not have enough resources to perform CA requests or when the CA is unavailable, the orchestration starts. It is managed on the cloud side. Once the CA is available, users' requests would not trigger the offloading technique. Table 2 summarizes the notations used in this study.

We Suppose containerized EA $\mathcal{A} = \{a_1, a_2, \dots, a_u\}$ include multiple workflow of microservices jobs listed as $\mathcal{J} = \{J_1, J_2, \dots, J_b\}$. We assume that the deployment workflow is organized in DAG structure $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_m\}$ denotes the node set and $E = \{e_1, e_2, \dots, e_n\}$ denotes the edge set. A node $v \in V$ represents the deployment of a single microservice orchestrated by K8s and an edge $e \in E$ represents the execution of microservice Docker container image and input data dependency between two microservices in deployment workflow. If v_i and v_j have container execution dependency, the edge $e_{ij} \in E$ will point from v_i to v_j .

We consider a set of N_i interconnected containerized data processing microservices: $i = \{i_s | 0 \leq s < N_s\}$; A set of data flows: $data_{ij}$, outflowing from v_i microservice, (denote as updo microservice) $v_i \in V$ to a downdo microservice $v_j \in V$: $E = \{(v_i, v_j, data_{ij}) | (v_i, v_j) \in V \times V\}$. A data flow streams sequentially with $Size_{ij}$ data elements $e = data_{ij}[\mathcal{X}]$, where $1 \leq \mathcal{X} \leq Size_{ij}$. We further denote a general data element as e to clarify the notation; A set of producers : $i_{src} \subset i$ generating data that prerequisite processing from the CA workflow. A producer has no updo microservices: $\bar{A}(i_s, src, _) \in E, \forall src \in i_{src} \wedge i_s \in i$; A set of end doers: $i_{snk} \subset i$ collecting and completing the CA output. A end doer has no downdo microservices: $\bar{A}(snk, i_s, _) \in E, \forall snk \in i_{snk} \wedge i_s \in i$.

When the deployment workflow with a DAG-organized job is submitted to the system, its microservices are encapsulated in container instances. We denote

Table 2 Description of notation used in this study

Notation	Meaning
G	DAG structure
V	Node set in DAG structure
$v \in V$	Deployment of a single microservice
v_i, v_j	Dependency container execution flow
$\mathcal{R}$	Edge nodes (servers)
E	Edge set in DAG structure
$e \in E$	Execution of microservice container image and input data
$\mathcal{C}$	Container sets
$\mathcal{A}$	Set of CA
$\mathcal{J}$	Set of microservices jobs
$Q_{ro,cq}$	Queue of waiting containers
N_i	Set of interdependent microservices for $\mathcal{A}_i$
$ F_i $	Set of single process of containers
l_t^r	Location sample value
rs_t^r	resource sample value
t_i	Slot time
λ	Provisioning time slot
$\lambda(t_i)$	time slot for offloading and orchestration
p_i	Number of N_i provisioned on edge nodes
$r_{N_i^{q_i}}(t)$	service rate in orchestrating $N_i^{q_i}$ at time (t)
h_i^k	Computation resource in orchestrating $N_i^{q_i}$ at time (t)
R_k	Pods created inside the K3S worker
k	Pods created to host container application
T_{max}	Maximum deployment delay of containers
CP_e	Capacity in resource to support the Pods
$B(t)$	Cost for availability of edge nodes at offloading time
$M(t)$	Functionality-aware cost at offloading time
$W(t)$	System cost
X_{ijk}	Binary variable to represent if workflow j of N_i on Pods k
F_{fij}	Variable to represent finish time of workflows j in A_i
F_{sij}	Variable to represent start time of workflows j in A_i

$\mathcal{C} = \{c_1, c_2, \dots, c_k\}$ as the container set, where $c \in \mathcal{C}$ denotes a single container, which is created in workflow with DAG structure.

We define $\mathcal{R} = \{r_1, r_2, \dots, r_l\}$, where $r \in \mathcal{R}$ denotes an individual K8s worker responsible for pod scheduling. For a K8s worker r , a container queue $Q_{ro,cq}$ is set to list the waiting containers, which is supposed to follow first in first out (FIFO) rule. The first containers queuing in different K8s workers can be run in parallel if they have no container or input data dependency. Otherwise, they must be run according to the workflow steps' dependency order. When the scheduling for large-scale CAs is submitted, the K8s controller will collect resource utilization on K8s edge workers. The objective is to minimize the completion time at a lower resource computation cost. When the K8s controller applies concurrent orchestration to schedule

CA service due to many user requests, all the K8s workers are fully loaded. If they are idle and their CA tasks are unscheduled, they wait in the orchestration queue of the pod created inside the K8s worker. The actual orchestration completion time of deployment workflow steps in provisioning CAs depends on the number of pods and containers in the deployment workflow template, container execution time, and edge cluster orchestration rate.

Functionality-aware offloading technique for scheduling containerized edge application

In this section, we formulate the container deployment workflow steps model in edge computing. We base our formulation on the fact that the orchestration depends on the number of workflow steps (created as microservices). Our proposed functionality-aware offloading technique implements two phases: pre-scheduling and computing edge nodes' resource consumption. In the pre-scheduling phase, an algorithm defines container-native workflow steps by composing them in container deployment workflows. Next, we propose an algorithm that computes the input decision on which edge nodes to start the scheduling of CA workflows. The input decision is critical for the functionality-aware controller to minimize the execution time while maintaining resource consumption. The architecture design of FAOFE is shown in Fig. 3.

Algorithm for pre-scheduling Argo workflow system

The K8s controller receives a deployment template that constructs workflow CA tasks. The deployment workflow template organizes the workflow CA tasks based on their respective dependencies. Moreover, the K8s controller receives the execution time and resource consumption information of edge nodes. The workflow CA tasks (in yet another markup language (YAML) template)

are heterogeneous in terms of the number of CA tasks, dependencies, and the amount of dataflow between each pair of dependencies CA tasks. Moreover, the K8s controller observes the order of execution of CAs tasks in each template.

Figure 4 demonstrates how his pre-scheduling phase works. Figure 4a represents two YAML templates (on K8s controller) with five and eight workflow CA tasks. The first template has one source CA task. The second template has three source CA tasks (represented in gray color). After collecting the source CA tasks, the *DependencyDAGExecutor* methods create a dependency relationship, a DAG_n^* with the DAG source CA task (embed inside a circle), as depicted in Fig. 4b. Next, the *BFS* algorithm applies on the DAG_n^* to specify the schedule number for each DAG CA task as depicted in Fig. 4c.

Algorithm 1 demonstrates how the pre-scheduling phase composes CA tasks in each template and accordingly creates a list of deployable of concurrent workflow CA tasks. For each template (submit to K8s controller), the workflows may execute several dependency. The *ContainerFinder* method finds all the workflow CA tasks of each dependency and stores them in Pod_n . This method checks the sequence execution order.

The *DependencyDAGExecutor* method receives the JB_n and Pod_n and creates a new workflow CA tasks by including dataflow output (as input arguments) and container image linked to each dependency job. This new workflow is called DAG_n^* . To achieve this, we create a dummy dependency root (called *DummyRoot_n*) and connect this CA tasks to all workflow of Pod_n resulting from the dependency execution. This allows us to run the breadth-first-search (BFS) algorithm over DAG_n^* . Therefore, from the *DummyRoot*, we can specify the execution order for each dependency job. The outcome result of the first loop

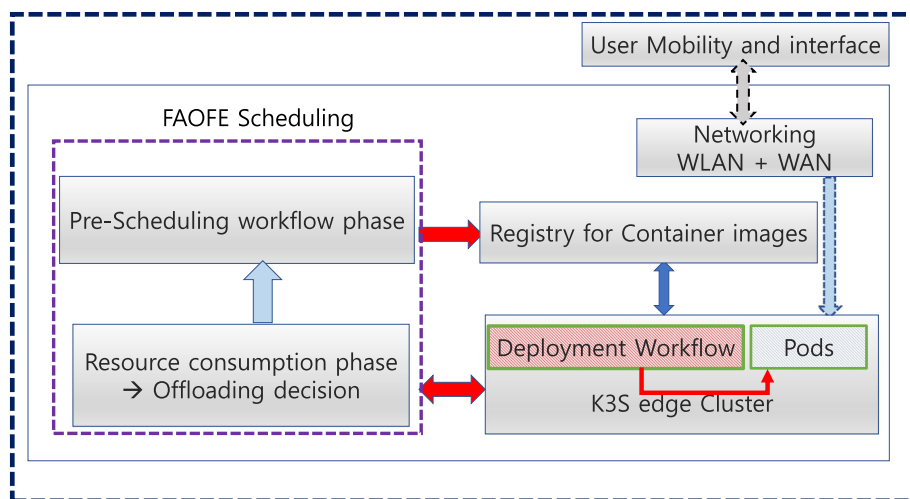
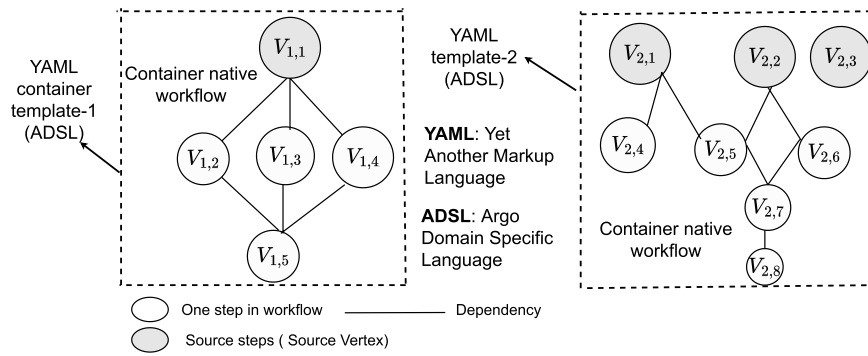
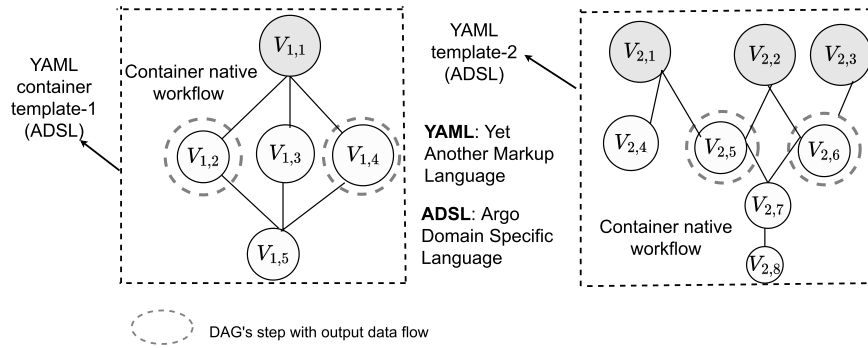


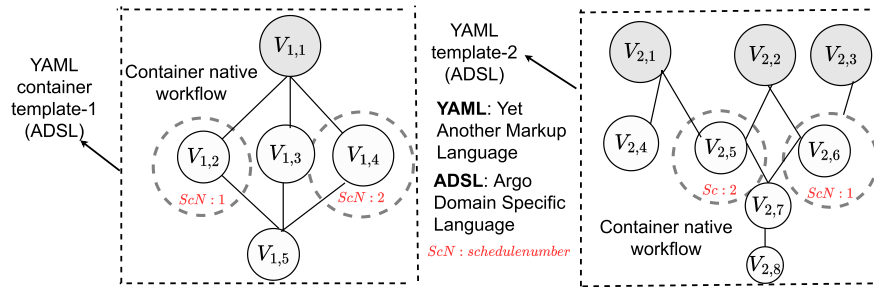
Fig. 3 The architecture design of FAOFE: Pre-scheduling workflow phase and Resource consumption phase. WAN: Wide area network; WLAN: Wireless local area network; K8S: Lightweight Kubernetes



(a) Different container-native workflows with DAG's dependency between them.



(b) Planning workflow steps with DAG's dependency in each template.



(c) assigning schedule number on multi-steps with DAG's dependency management.

Fig. 4 An example demonstrating the prescheduling of container-native workflows in the proposed offloading technique

of this algorithm provides a schedule number for the workflow CA tasks of each template.

The detail of *DependencyDAGExecutor* method is illustrated in Algorithm 2. The function to execute the dependency is shown in Algorithm 2. Asynchronously awaiting a message. The message case initiates the execution of a DAG. The collection associates a job to execute with a monotonically increasing index. To generate a topological structure representing the sequence in which the tasks are executed, the algorithm repeatedly goes through the task list and examines the dependencies among the other tasks. When a task completes, it should be queued up, executed, and finally removed from the agent state as an active dependency message case. Run the task function in the current context if the execution context captured has no value; otherwise, let

others know that a job has been finished, run it using the captured execution context, and publish the 'task completed' event. The event includes task information. The message case is used to add a task to be run in accordance with any dependencies, initiate the execution of the registered tasks, and then add a task, its dependencies, and the current execution for the DAG to be executed.

The function of BFS is shown in the Algorithm 3. Let the nodes with an indegree of 0 have any assigned number. After that, cut off the edges that emerge from them. We still have a DAG. There will be nodes with an indegree of 0. They can be assigned any number, but it should be a higher number than the initial set of nodes with an indegree of 0. Proceed in this manner, thus, a topological sort is produced.

Algorithm 1 Pre-scheduling CAs based Argo workflow engine on K8s cluster

Input: $Alg1$: status on worker node's resources
Input: CI : List of all container image transferred to the container registry
Input: $pipeline$: pipeline name
Input: env : configuration environment name
Input: JB : List of all jobs in the deployment Workflows
Output: FinalOrderList
 /* N : Number of workflows, JB_n : the n th job in the JB ,
 FinalOrderList: A 2D Arraylist in which tasks are executed
 in parallel */
Data: $N = |JB|$
for $n=1$ **to** N **do**
 $Pod_n = \text{ContainerFinder}(JB_n)$
 $DAG_n^* = \text{DependencyDAGExecutor}(JB_n, Pod_n)$
 $BFS(DAG_n^*, DummyRoot_n)$
for $n = 1$ **to** N **do**
 for $i = 1$ **to** $|JB_n|$ **do**
 $integerx = \text{CheckOrderNumber}(v_{n,i})$
 $FinalOrderList.get(x).add(v_{n,i})$

Algorithm 2 Dependency DAG execution

Input: $V = (i, E, i_{src}, i_{snk})$: CA workflow and data element
Input: $D = \{D_j | 0 \leq j < N_D\}$: Pods set
Output: $DLF[i_s]$ List of all dependency in the deployment Workflows
Data: Function $\text{DependencyDAGExecutor}(V, D)$
 $S \subset V \triangleright$ Microservices set with dependencies
 $S \leftarrow \emptyset, \forall i_s \in S \triangleright$ Initialize Microservice set with a dependency
 $Q \leftarrow \text{True} \triangleright$ Wait asynchronously for a message
for $i_s \in S$ **do**
 $chk \leftarrow \text{false} \triangleright$ boolean value used to notify when a dependency check
 completes $Agts \leftarrow \text{false} \triangleright$ boolean to track the tasks and dependencies
 if $D_j = \text{True}$ **then**
 if $Q = \text{True}$ **then**
 $Agts = \text{True}$
 $DLF[i_s] \leftarrow i_s$
 $Dchk \leftarrow \text{True}$
 else
 $D_j = \text{False} \triangleright$ Pods unavailable
 return $DLF[i_s]$

Algorithm 3 BFS

```

1: Input:  $DAG_n^*$  as an adjacency list, vertices  $i_{src}, i_{snk}$ 
2: Input:  $DummyRoot_n$ 
3: Function:  $BFS(DAG_n^*, DummyRoot_n)$ 
4: Set  $\mathbf{dist}[u] = \infty$  for all  $u$ 
5: Set  $\mathbf{prev}[u] = \text{NULL}$  for all  $u$ 
6: Create queue  $Q$ 
7: Set  $\mathbf{dist}[i_{src}] = 0$ 
8:  $Q.append(i_{src})$ 
9: while  $Q.size() > 0$  do
10:   Let  $u = Q.pop()$ 
11:   for each out-neighbor  $v$  of  $u$  do
12:     if  $\mathbf{dist}[v] = \infty$  then
13:       Set  $\mathbf{dist}[v] = \mathbf{dist}[u] + 1$ 
14:       Set  $\mathbf{prev}[v] = u$ 
15:        $Q.append(v)$ 
16:     end if
17:   end for
18: end while
19: Run and return topological structure along with  $\mathbf{dist}[t]$ 

```

Pod and container orchestration in pre-scheduling workflow phase

At system time t , an CA, A is scheduled. Once completed, a collection of interdependent microservices N_i that must be executed are provisioned at time $\lambda(t) = \sum_{i=1}^{N_i} \lambda_i(t)$. We assume that the number of containers provisioned in any time slot cannot be infinite. Therefore, we assume that there is a maximum value $\lambda_{max}(t)$ where $0 \leq \lambda(t) \leq \lambda_{max}(t)$. Therefore, we observe the following constraints at the slot time :

$$\lambda(t) = \sum_{i=1}^{N_i} \lambda_i(t), 0 \leq \lambda(t) \leq \lambda_{max}(t). \quad (1)$$

In case N_i are placed on different edge nodes, suppose p_i , is the number N_i provisioned on different edge nodes. Thus, $0 \leq p_i \leq N_i$ is a control variable representing the number of N_i placed on the edge node $R_i^{r_i}$. Under policy $p = (p_i, \dots, p_n)$, the load (creation) of N_i running on the $R_i^{r_i}$ is:

$$p = \sum_{i=1}^{N_i} p_i. \quad (2)$$

Execution time in pre-scheduling workflow phase

We assume that each container or workflow step within a container may have $|F_i|$ number of single-process balanced among $|N_i|$.

We assume that a dependent workflow step in F_i may have resource constraints concerning the CPU, memory, and network for its containerized single process. We model this by allowing a workflow step in F_i to service $F_i^{\text{CPU requests}}$ millicores for each of its single-process because it is not possible to use many of the CPU units available. Moreover, $F_i^{\text{mem requests}}$ and $F_i^{\text{mem limits}}$ specify the memory constraints of workflow steps in bytes.

The finish and start time of a workflow step is represented by Eq. (3). The finish time of a workflow step is equal to the sum of the start time and the time required to execute the workflow step at edge nodes.

$$Ff_{ij} = Fs_{ij} + \frac{c_{ij}}{N_i} * X_{ijk} \quad \forall i \in N_i, \quad j \in F_i, \quad k \in R. \quad (3)$$

Edge cluster orchestration rate in pre-scheduling workflow phase

The orchestration rate of $N_i^{n_i}$ container provisioned at time slot t is $r_{N_i^{n_i}}(t)$. Because the variation in computing resources of each container and its associated workflows steps are important, we assume that the $r_{N_i^{n_i}}(t)$ is randomly employed. For each pod, we denote $N_i^{n_i}$ as a computation resource h_i^k .

As per the above description, we calculate the following: (i) the number of offloading requests accepted by the system $\gamma(t)$; (ii) the number of interdependent containers $N_i^{n_i}(t)$ to A_i of CAs; (iii) the number of offloading requests executed for each set of N_i at orchestration rate $r_{N_i^{n_i}}(t)$. All must satisfy the following constraints:

$$\gamma(t) = \sum_{i=1}^{N_i^{n_i}(t)} r_{N_i^{n_i}}(t). \quad (4)$$

CP_e denotes the capacity of an edge node. When CP_e diminishes, a new value of N_i is provisioned per available resources at the edge node. We define the budget of the entire EC infrastructure as B . The budget of the EC infrastructure must be set such that a set of active edge nodes can support all CAs. For this purpose, we define the budget of available EC nodes as the average capacity and at time t the number of assigned microservices of CA, $|S|$,

$$B(t) \geq \left\lceil \frac{|S|}{\sum_{N_i} CP_e |\gamma(t)|} \right\rceil. \quad (5)$$

For any orchestration time slot t , we denote the orchestration service rate as $M(t)$. Therefore, the overall orchestration service rate and the number of N_i associated with the CAs $A(t)$ have a linear relationship:

$$M(t) = \gamma(t) \cdot A_i(t), \quad (6)$$

In summary, we define the overall service rate of deploying CAs during each offloading time slot as a combination of the orchestration service rate and the computation on the availability of EC nodes. Denoting the overall service rate of offloading service deployment in the orchestration time slot t , thus, we obtain:

$$W(t) = \alpha_1(c_p(t)) + \alpha_2 G(t), \quad (7)$$

where α_1 is the orchestration service rate from Eq. 4. α_2 (Eq. 5) represents positive weight parameter on the efficiency of edge nodes' computation resources. We also use the following binary decisions in the model. $a_n \in [1, 0]$ decides whether an edge node $r \in R$ has submitted its edge node's computation resource utilization.

Computing of scheduled workflow

Creating some containers incurs some computing resource consumption on edge nodes. The goal is to decide which scheduled workflow to serve so that the sequence of them is executed with the minimum possible cost of resources of edge nodes.

Consider scheduled workflow is running in container runtime engine (Docker), the completion time is defined as:

$$T_i = \begin{cases} T_i^{nod-1}, & \text{CNT1-creation} \\ T_i^{nod-1}, & \text{CNT2-creation} \end{cases} \quad (8)$$

For each container, the completion time is defined as follows;

$$T_i^{CNT1} = R_i^{nod-1} + W_i^{nod-1} + \frac{D_i}{B_{nod-1}} + RTT_{VM}. \quad (9)$$

where CNT1-creation has limited resources, and the initiation of container should include the waiting time W_i^{VM} . As a result, the completion time aggregates the scheduling execution time R_i^{nod-1} on $v_{nod-1} \in V$, waiting time W_i^{nod-1} , and exchange data time $\frac{D_i}{B_{VM}} + RTT_{nod-1}$.

The completion time denotes the time at which the edge server completes instantiating the containers as defined into the host at the edge, the completion time is calculated by:

$$T_i^{host} = R_i^{host} + \frac{D_i}{B_{host}} + RTT, \quad (10)$$

where R_i^{host} is the service rate in orchestrating CNT1-creation, D_i indicates the size of the microservice image retrieved and assigning processes, B_{host} is the bandwidth, and RTT (round-trip time), the network delay. In general, the CNT1-creation is considered to have an adequate resource capacity.

To formalize the completion time on heterogeneity edge nodes, we define an allocation vector $V = (v_1, v_2)$ to indicate whether the scheduling of workflows is assigned to a container running in the cluster of edge nodes. In this vector, V_1 denotes whether the scheduling is executed on an edge node-1, and V_2 on edge node-2; value 1 represents yes; otherwise 0 represents no.

$$S_{pi} = R_i^{CNT} / AT_i. \quad (11)$$

Then, for all scheduled workflow in T , the objective of elastic computing is to maximize the following expression:

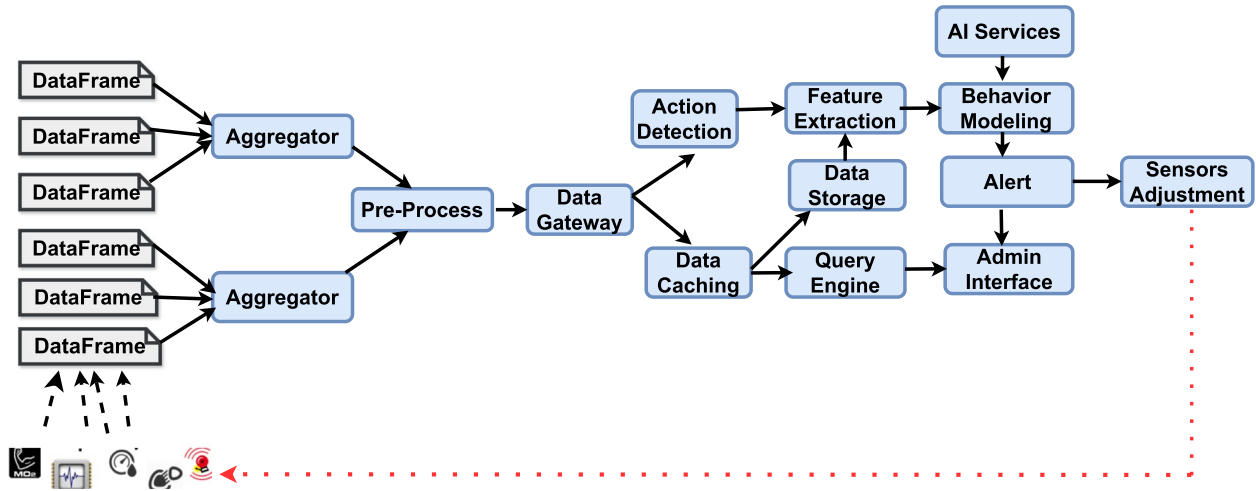


Fig. 5 Internet of Things (IoT) application example: A common IoT system workflow

$$\bar{S}_{pi} = \left(\sum_{i \in J} \frac{R_i^{CNT}}{AT_i} \right) / A_i, \quad (12)$$

the average elasticity fast configuration, where A_i represents the number of containerized services within the workflow. Specifically, for each scheduled interdependent containers N_i , S_{pi} should be greater than 1, which means that the computation time should improve container creation performance when scheduling workflows based on DAG.

IoT application illustrated as a DAG workflow

Figure 5 describes an IoT application illustrated as a DAG workflow, where each vertex within the workflow represents an IoT task. Generally, a complete IoT workload is made up of several IoT tasks that can be deployed in separate containers. Each container is coupled to create a function chain that best meets the needs of the application, running on either physical or virtual machines. As a common IoT application,

it is divided into numerous separate but cooperative microservices. Some subsystems can specifically perform tasks such as alerts for abnormal events, real-time data analysis, and remote monitoring. End users can be alerted immediately to take necessary action if anomalous behavior is detected via integrated sensors that generate data. Data aggregators can receive this data continuously. After that, the pre-process microservice computes the proper level of data to be transferred to the data gateway controller, and sends it when an action is detected or performs caching for later retrieval query. The Action detection microservice execute feature extraction that the AI microservices will use to report the behavior to different sequences of microservices. Any alert view through the admin interface will allow finally the execution of sensors adjustment microservice. Table 3 depicts 6 different workflows from the sensors generated data to different sequences of microservices. The seventh workflow has a sequence that starts from AI services and proceeds to the sensors' adjustments or the admin interface.

Table 3 Workflows from the src to the last tasks of general IoT use case application

Workflows	Sequence of Workflows
1	dataframe - aggregator - preprocess - Data Gateway - Action detection - Feature Extraction - Behavior Modeling - alert - sensors Adjustment
2	dataframe - aggregator - preprocess - Data Gateway - Action detection - Feature Extraction - Behavior Modeling - alert - admin Interface
3	dataframe - aggregator - preprocess - Data Gateway - Data Caching - Query Engine - admin Interface
4	dataframe - aggregator - preprocess - Data Gateway - Data Caching - Data Storage - Feature Extraction - Behavior Modeling - alert - sensors Adjustment
5	dataframe - aggregator - preprocess - Data Gateway - Data Caching - Data Storage - Feature Extraction - Behavior Modeling - alert - admin Interface
6	AI services - Behavior Modeling - alert - sensors Adjustment
7	AI services - Behavior Modeling - alert - admin Interface

The pre-scheduling phase uses an execution model to schedule the dataflow processing application while taking into account synchronous microservices communication (see [System model](#) section). After the application's DAG is constructed, it first determines the dependencies among the microservices (Algorithm 2). Next, it applies the execution time objective algorithm, which considers the finish and start times of a workflow step. The BFS algorithm (Algorithm 3) gathers data from all inputs for the current task, then executes the workflow for the current task with input values in scope. It sets the resulting data to the task data (constructing something akin to calculated data, where the output pin can only accept a portion of the data). It then proceeds to the next output-connected tasks. The aggregator task gathers sensor data at predetermined intervals (e.g., 15–30 frames per second) and combines them into one sizable data frame.

Computing edge worker nodes' resource consumption phase

The computing edge worker nodes' resource consumption phase aims at analyzing and providing the next offloading decision to edge servers. The function uses historical data collected from edge node's computation resource utilization.

The flowchart of the functionality aware controller is shown in Fig. 6. The controller uses a store component to collect historical resource usages. The functionality-aware store component uses a metric for clustering edge node to provide a list of all available edge nodes in the cluster that can perform the desired offloading scheduling. The executor component retrieves locality and available edge node time series data and starts a background thread to process the offloading decision by comparing with a query on functionality-aware store that retrieves the output target. Then, it allows the K8s controller to process workflows. Finally, the K8s controller extends the functionality aware offloading controller and handles the orchestration at an edge infrastructure level.

Algorithm 4 presents the proposed computing edge worker nodes' resource consumption phase. At the time offloading slot, t , the analysis of edge nodes' information about resources is sampled according to *ResourceOrderList*. Here, the values stored as l_t^r and rs_t^r representing the values sampled by the algorithm from the edge nodes r at the offloading time slot t . Thus, there are the vector $X_t^r(resources) = [rs_t^r, rs_{t-1}^r, \dots, rs_{t-\phi+1}^r]$ containing ϕ of the last observations for each r edge node. The algorithm outputs the *finalResourceCost*. It is the value of the cost on resource consumption after scheduling the workflow steps. *StepsExec(EdgeMetric)* method receives the offloading

Algorithm 4 Computing edge nodes' resource consumption

Input: HD : List of all historical data from edge nodes
Input: *ResourceOrderList*: The 2D ArrayList containing all edge nodes' resource consumption
Input: $X_t^r(location)$: historical data on edge location
Output: finalEdgeConfigs, finalResourceCost
 /* N : Number of edge nodes, HD_n : the n th time series data, Q : number of all schedules workflow steps, *EdgeMetric*: A global 2D list arrayList in which each row stores the offloading configuration of workflow steps, *finalEdgeConfigs*: A 2D ArrayList storing obtained edge node's configuration for each template including workflow steps, *finalResourceCost*: An array to store resource consumption cost for each template */
Data: *EdgeMetric* := null
for $i=1$ **to** Q **do**
 | *EdgeMetric*.get(i)(*ResourceOrderList*.get(i))
 | *finalEdgeConfigs*=StepsExec(*EdgeMetric*.get(i))
for $n = 1$ **to** N **do**
 | *finalResourceCost*[n]=Cost(*finalEdgeConfigs*)

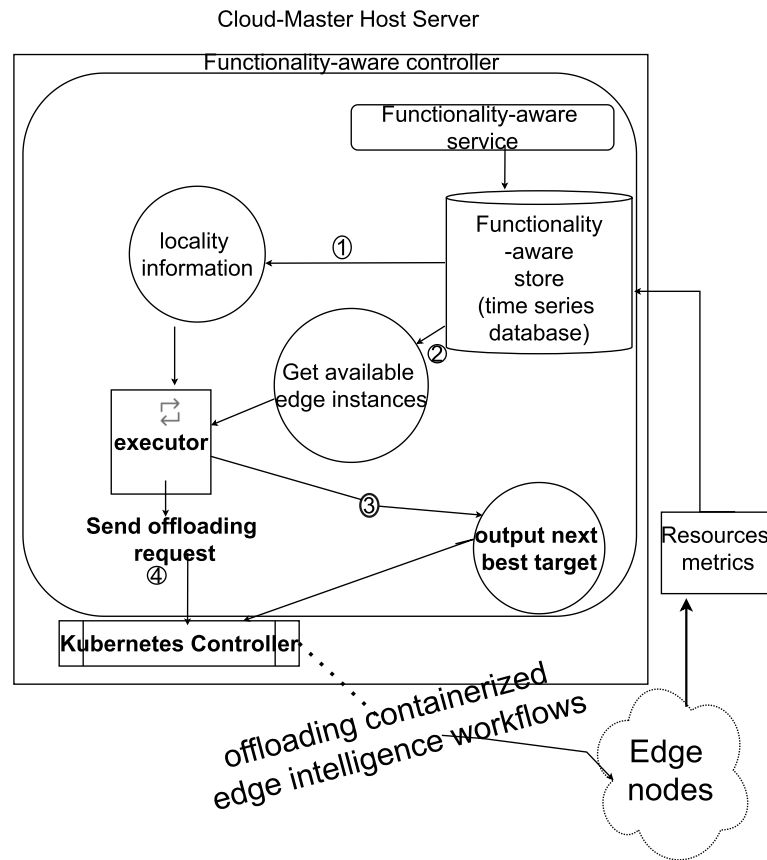


Fig. 6 Overview of the functionality-aware controller

configuration of all scheduled workflow steps, organizes the workflow steps in each workflow template, and stores them in a 2D Arraylist called *finalEdgeConfigs* so that each row represents one workflow template. For the offloading of all scheduled workflow steps in each template, the *finalResourceCost(finalEdgeConfigs)* method calculates the resource consumption of each workflow template (based on the

provided configuration) on edge nodes. The algorithm has some limitations for large deployments. In fact, workloads might need to be distributed across multiple K8s clusters. The algorithm lacks the definition of Workflows based on pre-defined events to schedule workflows across different clusters. This setup would allow for greater scalability and fault tolerance which will be studied in our future work.

Algorithm 5 Stores data related to underutilizing or overutilizing resources

Input: Host Overload/Under-load: assess the resource status
Input: hosts that are underutilized U
Output: log into InfluxDB resource status and CPU loads
Data: initialization U
for *host identified* **do**
 host ID _success.add_listener *host ID*_success(container module_time, consolidated list_length) Overload Host List_CPU_averages()
 if *error* **then**
 *host ID*_failure.add_listener *host ID*_fail(resource_type, exception)
 Overload Host List_CPU_averages|()
end for

The function of storing resources in the time series database is given by the procedure 5. The following components are implemented to store edge node resource model in the computing edge worker nodes' resource consumption phase (Fig. 7).

- **Host Overload/Under-load:** This component is employed to regularly assess the resource status, determining whether the host is either underutilizing or overutilizing resources. The primary objective is to pinpoint hosts that are underutilized, and once identified, the host ID is sent back to the container module along with a consolidated list of all containers currently running on that host.
- **Overload Host List:** This component stores data related to all hosts that exhibit or trigger an overload status.

Evaluation

We conducted extensive experiments to evaluate the performance of our proposed algorithm. Then, we present the simulation evaluation of the proposed scheduler. The proposed offloading technique is evaluated together with existing workload scheduling algorithms.

Experimental setup for actual trace

In this subsection, we first discuss the experimental setup. The preliminary experiments allowed us to collect practical data traces and parameters used in the simulation. A list of microservices and their Docker images is provided. The scenario involves a health mobile system that uses wearable devices attached to subjects to classify their activities. The application focuses mostly on data processing and inference services since we use an open dataset.

Experimental setup

To validate our containerized edge inference service and collect data for the simulation setup, we carried out experiments in our research laboratory. For edge devices, we used Raspberry Pi 4, and the cloud core was set up on Amazon Web Services.

The Raspberry Pi 4 is an ARM device (armhf). It runs Ubuntu Arch Linux ARM aarch64 for Raspberry Pi with kernel version 5.8.7-1-ARCH on the configuration, which includes 4 GB of RAM and a quad-core Cortex-A72 (ARM v8) clocked at 1.5 GHz.

K3s is selected because it is a lightweight variant of Kubernetes, making it more suitable for edge scenarios. It features mutual master-worker networking, and it consumes fewer edge resources.

The setup of the K3s edge node cluster illustrated in Fig. 8 is discussed as follow:

- We configured the Machine-Controller-Manager (MCM) [35] to cooperatively manage the lifecycle of the worker machines. The K3s master server was set up on an Amazon Web Services (AWS) instance.
- By using the Terraform controller [36] for Kubernetes, we were able to connect the cloud master host to the edge workers.
- The cloud node has a public address, while the edge nodes do not hold public IP addresses. To address this issue, we installed Tailscale to establish a lightweight Peer-to-Peer (P2P) VPN connection. This connects the Tailscale network [37] with the cluster. Once Tailscale is operational, the worker K3s in the cluster has network connectivity to the K3s API server, making it easy for each worker K3s to join the cluster.
- On each node, we generate the corresponding configuration scripts to ensure that each node has access to

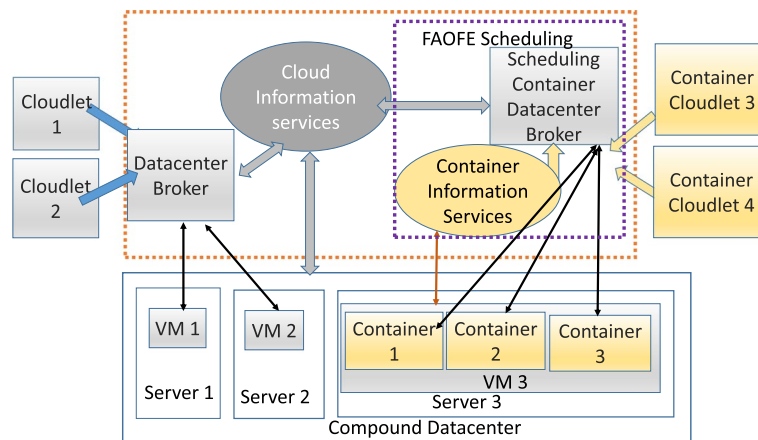


Fig. 7 Our approach design to perform simulations of the proposed scheduling in ContainerCloudSim

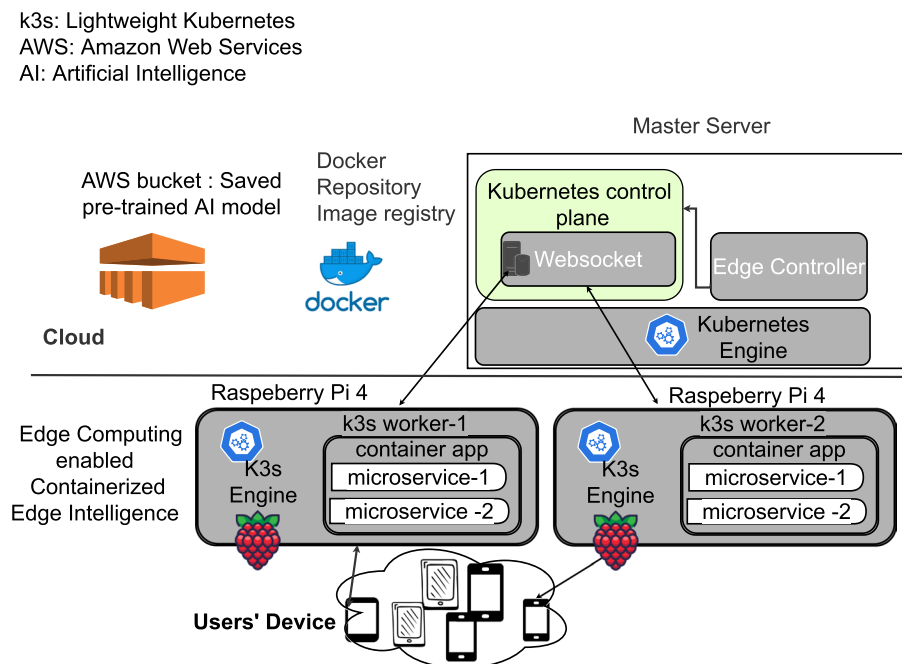


Fig. 8 Overview of the experimental setup for k3s cluster, two Raspberry Pi as workers node at edge and cloud core

all other nodes in the cluster. Since K3s runs natively in Docker containers, we use Docker mode to initialize the K3s server, enabling it to access the Docker images.

- We install K3s on each agent, specifying the IP address of the master node server and the token obtained from the master node server during installation. All worker nodes can then properly connect to the master node server.

Docker image and containers applications on edge node worker

The docker image of the pre-trained AI model and its dependencies scripts are wrapped in container image and stored to our docker image repository. The Table 4 lists those images in our repository [38]. The pre-trained AI model is stored in cloud storage; in this case, the AWS bucket was selected to store the pre-trained AI model. After pulling the AI inference image, the pre-trained model is downloaded and stored locally within the container pod

The workload in the pre-defined workflow using Argo K8s native was performed by calling the pre-trained model (downloaded and stored locally) and then executing the prediction microservice running on Raspberry Pi. We simulated many simultaneous inference requests for the activity prediction service. All edge workers' resource usage is captured as time series and stored in InfluxDB for further analysis. The log entries are monitored through

metrics provided in the Grafana monitoring framework [39]. The logs on resource utilization are used as input to the functionality-aware controller for the host and container's resource consumption. The goal of the analysis is to provide the agent at the functionality-aware controller in the cloud node with the capability to scale the workload of workload inference requests to meet the acceptable inference latency time. The collected time-series data in InfluxDB are normalized and reformatted. The source code for building the software images is referenced in [40].

Figure 9a shows the average response times for the new mHealth dataset [41]. The dataset is uploaded to a processing server deployed across cloud, and edge computing infrastructure. In this context, the response time ranges from creating a task from an mHealth dataset to the mHealth inference AI model service server receiving the response. Among the worker nodes at different layers, edge worker nodes exhibit the lowest response time of 850 s, while cloud and edge workers have response times of 3500 s and 2000 s, respectively. Figure 9b illustrates the average loading waiting time of the mHealth model after the container runtime deploys the containerized application. When worker nodes operate at the edge, the average loading time for the TensorFlow API and integration loading Keras APIs/starting Flask server is approximately 1700 s and 2700 s, respectively. The cloud exhibits shorter loading time than the mHealth model when the image is initially pulled from the repository located in the cloud.

Table 4 Our docker hub image repository: image, OS/arch and their compressed size

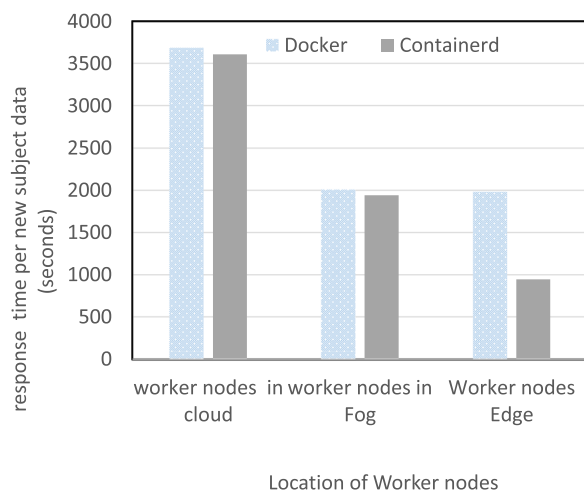
Image name	OS/Arch	Compressed size	Task
nkenye1982/mhealth_ai_service:1.0	linux/amd64	1000.03 MB	AI service
nkenye1982/mhealth_ai_service:armarch	linux/arm	1000.27 MB	AI service
nkenye1982/grafana-graphite:1.0	linux/amd64	404.4 MB	Monitoring
nkenye1982/locusttask:1.0	linux/amd64	359.9 MB	Loading requests
nkenye1982/todo-proxy:1.0	linux/amd64	42.69 MB	Proxy
nkenye1982/todo-view:1.0	linux/amd64	6.74 MB	View Interface
nkenye1982/todo-api-gateway:1.0	linux/amd64	23.96 MB	API
nkenye1982/todo-search-service:1.0	linux/amd64	24.56 MB	Search service
nkenye1982/todo-cache-ingestion:1.0	linux/amd64	23.11 MB	Data Caching
nkenye1982/todo-storage-ingestion:1.0	linux/amd64	23.17 MB	Data storage
nkenye1982/todo-cache-service:1.0	linux/amd64	23.37 MB	Caching Query
nkenye1982/todo-storage-service:1.0	linux/amd64	23.58 MB	Save service
nkenye1982/todo-postgres:1.0	linux/amd64	111.45 MB	Postgres database
nkenye1982/todo-search-ingestion:1.0	linux/amd64	24.14 MB	Search query

Evaluation on basic workload scheduling model

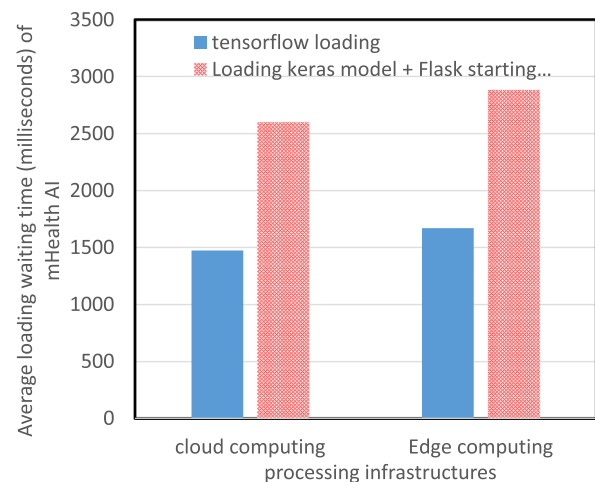
The simulation was performed using ContainerCloudSim [42, 43]. Recently, CloudSim 4.0 has been introduced with features and extensions for container simulations [42]. Our simulation scenario in Fig. 7 has a data center that accommodates containers and computes the FAOFE scheduling scheme. We extended ContainerCloudSim classes implemented in [43]. In ContainerCloudSim Container Cloudlets are submitted at the start of the simulation time to the Scheduling Container Datacenter Broker then the Broker assigns Container Cloudlets to containers directly. The resource consumption of containers is provided to Container Information

Services. It is a registry for all virtual machine resources that provides registration, orchestration time of Container Cloudlets, and discovering capabilities. In the pre-scheduling phase that follows the DAG structure, we use attributes presented in [43] for application-connected Container Cloudlets. The attribute PerquisiteCloudlet registers an ArrayList of ContainerCloudlets that must complete execution before the given container cloudlet enters the pipeline deployment workflow steps to simulate the dependencies between container cloudlets belonging to the same application.

We evaluate the proposed offloading orchestration scheduler against the basic workload scheduling and



(a) Average response time for new mHealth data on worker nodes.



(b) Average loading waiting time of containerized EI on cloud and EC.

Fig. 9 Average response time and loading new containerized EI service

K8s-based orchestration scheduler. In basic workload scheduling, we compare the performance of FAOFE with the following two types of workload scheduling strategies in mobile edge computing:

- The SJF [44] scheduling algorithm prioritizes short jobs first, selecting the next jobs with the shortest estimated running time from the waiting queue to execute them first.
- Round-robin [45] assigns a fixed time slot for the scheduler by defining the length of time for each task execution before to executing the following task.

In Kubernetes-based scheduling, the FAOFE scheduler is compared to the Kube-scheduler [46] and the infrastructure-aware scheduler [27]. Kube-scheduler is a default scheduler in K8s platform and is responsible for making decisions about where to place pods in the cluster. The main goal of an infrastructure-aware scheduler is to make informed decisions about where to place pods. It takes into account not just factors such as resource needs and affinity rules but also considers the inherent characteristics of the underlying infrastructure, including network-related considerations.

The results of four simulations are presented. In the first simulation, we carefully examine the workflows of each CA microservices at each offloading orchestration time. The second simulation discusses the offloading/scheduling service time of the proposed method in accordance with containerized microservices implemented for the CA. The third simulation shows the average of container resource utilization created on an edge node as the offloading/scheduling load increases. Finally, the fourth experiment discusses the delay of the overall system expected to handle scheduled containerized microservices for a AI inference service with pre-trained AI model. The images of these containers are hosted to the author's Docker container account [38].

First simulation: the service rate of the CAs at each offloading orchestration time

The simulation evaluates the service rate of the CAs at each offloading orchestration time. The service rate on the bandwidth in this experiment is between 10 and 100 Mbps traffic load for CAs. The more parallel offloading of the orchestration of CAs, the more data can flow through. This is because we excluded the effect of the network latency (physical limitations of the network device, such as the router or wireless frequencies being used) on the overall system. The constant traffic of 10 Mbps is injected into the network for 100 s. Figure 10a represents the service rate for offloading the CAs from the cloud-to-edge nodes. In this experiment, since the

traffic to all the edge nodes is equal, the bandwidth to all the edge nodes is equal and has the highest bandwidth of 100 Mbps for all 100 s.

Second simulation: offloading service time of the proposed method in accordance with containerized microservices

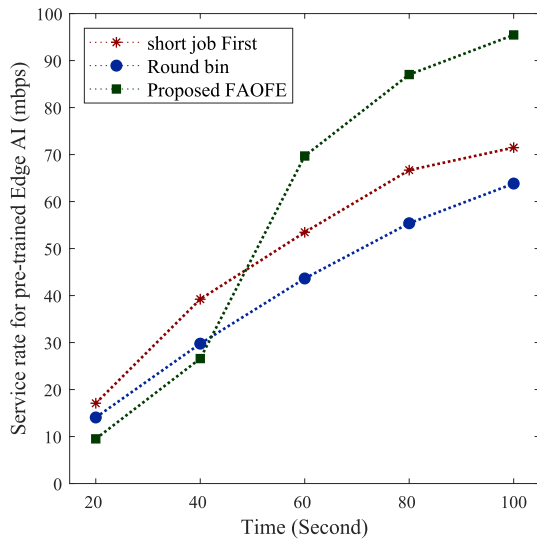
Figure 10b illustrates the service time to orchestrate containerized AI [38] model in according to the number of containers-based microservices. Our proposed FAOFE achieves a service time of orchestration about 2 s for an orchestration slot time range of 20–200 s.

Based on this, our proposed algorithm consistently achieved optimal service time performance, consuming 0.4 s, 0.52 s, 1.7 s, and 1.52 s service time at orchestration slot times 20, 80, 120, and 160 respectively. This is because our proposed FAOFE algorithm uses the pre-knowledge of resource utilization of the edge nodes during offloading scheduling of containerized EA. In our proposed FAOFE algorithm, the functionality-aware prediction function was used to search for the best resource utilization on edge nodes, ensuring the best possible scheduling by retrieving locality and resource utilization information. This reduces service completion time regardless of the number of containers to implement CAs. Considering the round-robin algorithm with an equivalent service time of approximately 1.5 s. The SJF technique has the highest service time compared to the two techniques, but this service times decreases to almost 1 s when the orchestration slot time reaches 160 s. This means that the short job execution of offloading based on the priority injected reduces the service time of the SJF when the orchestration slot time increases.

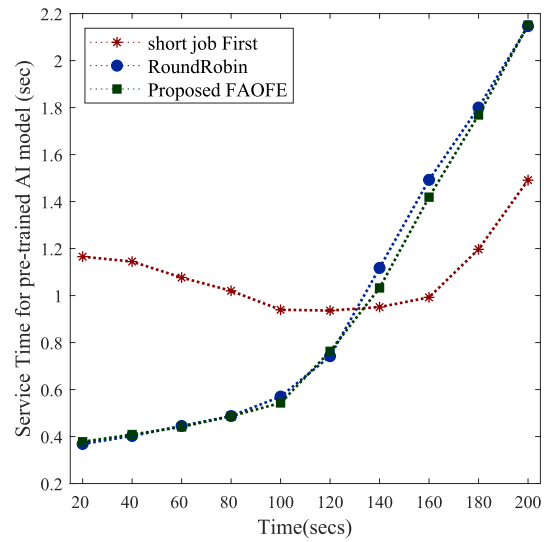
Third simulation: average of container resource utilization as the offloading load increases

Figure 11a illustrates container resource utilization performance versus different workloads. As expected, we can observe that number of container resource utilization for the proposed, round-robin, and SJF increase as the offloading load increases. The SJF has lower resource utilization because it prioritizes jobs with short completion times, which demand fewer computation resources. In addition, resources for containers running on edge node for FAOFE are lower than those for the round-robin technique when the offloading load is at a moderate level between 40 and 180 s. At a lower level of offloading load (less than 40 s), CAs are scheduled with lower resource utilization in FAOFE. When the workload is not higher than ≤ 100 , all approaches achieved lower container resource utilization.

Overall, FAOFE can effectively achieve performance goals in terms of resource utilization to run CAs on edge hosts within a certain offloading load level. However,



(a) Service rate for containerized pre-trained Edge AI (mbps) and orchestration time.



(b) Service time for pre-trained AI model by number of microservice per container.

Fig. 10 Service rate and time for scheduling CAs related to AI inference service on K8s lightweight

resource utilization in FAOFE is no longer regulated (to a value of 20 percent) when the offloading load increases above 120. This is because the edge physical resource in the system implements an elasticity computing algorithm as more containers with different resources consumption degrees during deployment are scheduled.

Fourth simulation: the overall delay

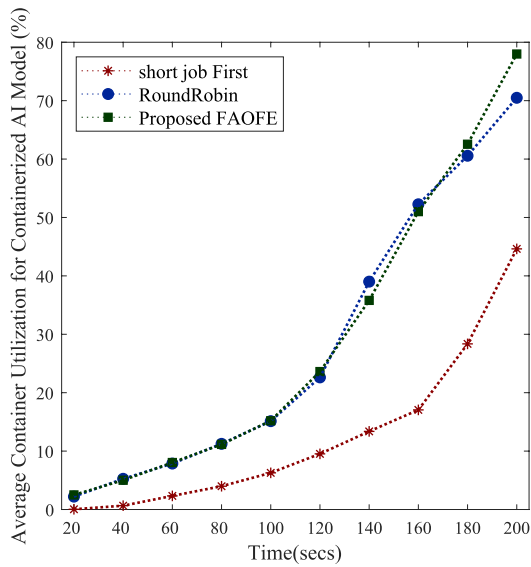
Figure 11b shows the system latency versus the number of end users abstracted as cloudlets. We assume that tend users load data for AI inference. When the data set is large enough to be processed by the nearest edge node, the proposed scheduling drops current created pods, then forwards the failure to the functionality-aware monitoring, which offloads the CA service to a new edge node server with sufficient resources. This brings additional transmission delays from the current edge node to the next available edge host. As a result, the system latency increases with the number of end users. Unlike the SJF and round-robin methods, which exhibit delays of roughly 50 and 35 seconds, respectively.

Evaluation on K8s-based scheduling model

The simulation focus on better distribution of CAs by scheduling them on the edge node cluster. The scheduling of containers on the edge node cluster and their proper placement are evaluated. Our experiments are focused on K8s and the scheduling of CAs on its cluster with a focus on CA performance in the edge node cluster.

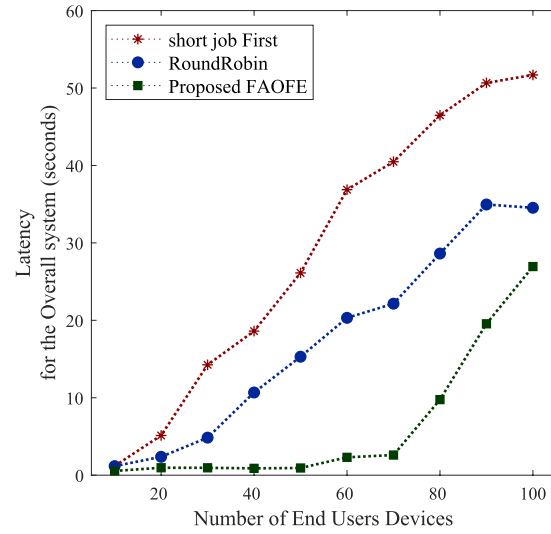
First simulation: the network delay of the CAs for different offloading times

The proposed method in evaluating the service delay of CAs. In this regard, CAs are built with the following microservices: pre-trained, pre-processing, and inference container. They are had different offloading time slots (ranging between 20 and 200 s). Figure 12a depicts the average network delay of the system. As illustrated, for the proposed method, when the containerized workflows are orchestrated, the network delay is slightly low (0.6 s) compared to WAN delay of 1.5 s (as shown in Fig. 12b) at simulation time of 200 s. The orchestration of CA with Kube-scheduler decreases with the simulation time and ends with a smaller network delay of 0.8 s. The network delay of CA service, co-executed with FAOFE controller increases with the simulation time because the computation of offloading decision adds processing time. Compared to other methods, if more offloading requests are submitted in the system (after 120 s), the infra-aware scheduler technique performs better than the Kube-scheduler. However, for wide area network scenario (WAN delay Fig. 12b) where network policies are required, the Infra-aware showed high latency than Kube-scheduler because it is customizable, implementing customized and extensible network-aware scheduling policies. Infra-aware scheduler shows same performance as the proposed FAOFE scheduler. The FAOFE scheduler technique efficiently distributes the orchestration of containerized workflows among edge worker nodes. Therefore, the proposed method uses the availability of nodes to begin the computation of resource consumption of edge worker nodes before the offloading process starts;



(a) Number of container orchestrated for each offloading load to the user AI requests.

Fig. 11 Resource utilization and latency of scheduled containers



(b) Latency for accessing CA services scheduled from virtual end users .

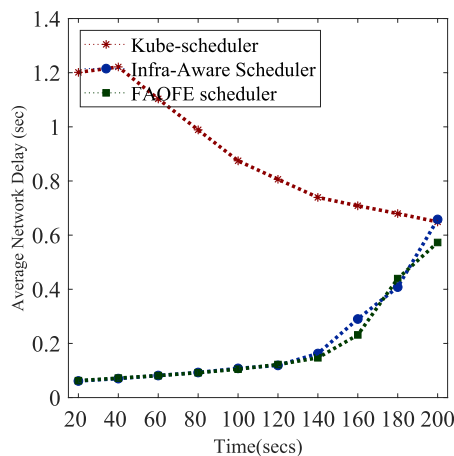
however, as the workload increases, the parallel offloading affects the computation of running edge worker nodes at the FAOFE controller, then increases the network delay.

The potential network bottleneck impact the operation of the proposed model in scheduling delays and communication delays. The scheduling delays include delay by computing the analysis of the cluster's nodes, including their resource utilization, availability, and networking capabilities. The scheduling delays lead to slower decision-making and pod placement. If there's a network bottleneck between the the K8S controller and offloading

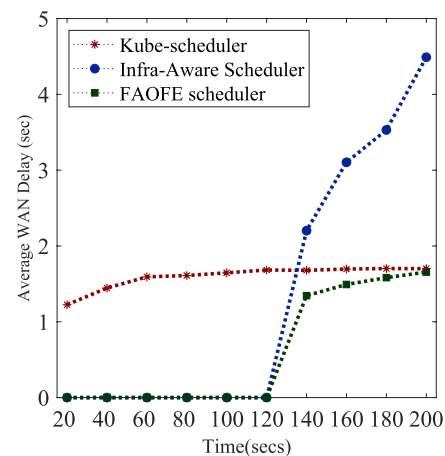
controller, it can result in increased latency for scheduling operations.

Second simulation: the number of Pods expected to handle scheduled CA randomly generated

Figure 13a illustrates the performance of our proposed FAOFE scheduling technique. The total time was set to a range from 20 to 200 s, and the number of pods running the container applications is fixed within the range of 0 to 15. We assume that the number of pods increases by five for every 70 s iteration. We assume that a pod (a group of one or more containers) is configured to run a



(a) Average network delay (sec).



(b) Average WAN delay (secs).

Fig. 12 Network delay against Kube-scheduler and Infra-aware scheduler

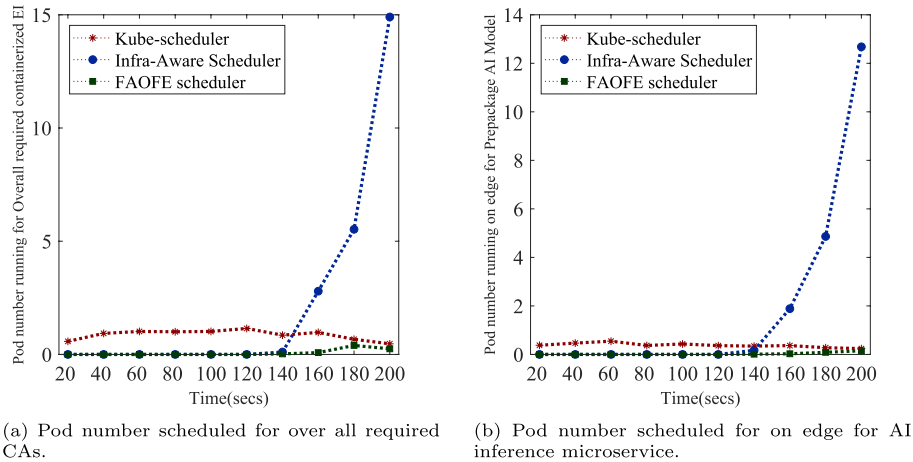


Fig. 13 Pod number to provision the containerized pre-trained Edge AI microservice

single container, known as the “one-container-per-pod” model. Figure 13b illustrates the number of pods created, indicating that even though the maximum number of pods to be instantiated for all applications could exceed 10 pods. For the infra-aware scheduler technique, the number of pods increases every 140 s. After 140 s, the load of scheduling increases, prompting the creation of new pod replicas. Then, load distribution is initiated, and so is load balancing between pods. The Kube-scheduler technique is designed to create one or more pods and will continue to retry the execution of pods until a specified number of them successfully terminate. This is why the “one-container-per-pod” model fails. With more simulation time, the FAOFE technique ensures resource availability for parallel processing, making the final number of Pods for the completion times constant at one pod, until 200 s. At 180 s, the FAOFE algorithm instantiates the CA in one pod.

Third simulation: overall processing time of the offloading decision on edge

Figure 14a illustrates the processing time of edge nodes’ resource utilization collected at the edge of the network. The output serves as the input to the offloading controller. As expected, kube-scheduler has lower processing time because it collects pods metrics with short completion times, which demand fewer computation resources. Furthermore, the computation of resources for containers running on the edge node for FAOFE is lower at the beginning because they are few containers running and the workload of AI tasks is low. The processing time is lower than for the infra-aware scheduler technique when the offloading load is at a moderate level between 40 and 180 s. At a lower level of offloading load (less than 40s), CAs are scheduled with lower resource utilization in FAOFE. When the simulation

time is ≤ 100 s, Kube-scheduler approach achieved lower processing offloading decision.

Overall, FAOFE can effectively achieve performance goals in terms of deciding the next edge worker nodes to offload CAs but the overall processing time increases as offloading requests increase in the system.

Figure 14b shows the system latency for computing the offloading decision in the cloud. We assume that the FAOFE controller collects data from edge worker nodes. When the data set is large enough to be processed by the nearest edge node, the creation of the CA fails, then the system forwards the failure to the functionality-aware monitoring, which computes and then offloads the CAs to a new edge node server with sufficient resources. This brings additional transmission delays from the current edge node to the next available edge host. Consequently, the system processing time based on the cloud infrastructure increases with the number of available edge worker nodes. This is in contrast to the Kube-scheduler approach, with stable processing time on the cloud (approximately 0.12 s) as the number of requests to orchestrate CAs increases (simulation times).

The FAOFE scheduler is compared to the joint task scheduling and containerizing for efficient edge computing (JTSC) [47] and the Robust-PAC time-critical workflow offloading [48] in a simulated scenario based on task execution in containers with their workflows modeled as DAG in an edge network topology.

The metric used for comparison includes service level agreement (SLA) deployment latency. For simplicity, the simulation only considers network utilization for provisioning containerized applications, from the registry to the worker or edge nodes. We ignore other communications that may occur in a real network. The official Docker Hub registry is in the cloud, where all nodes

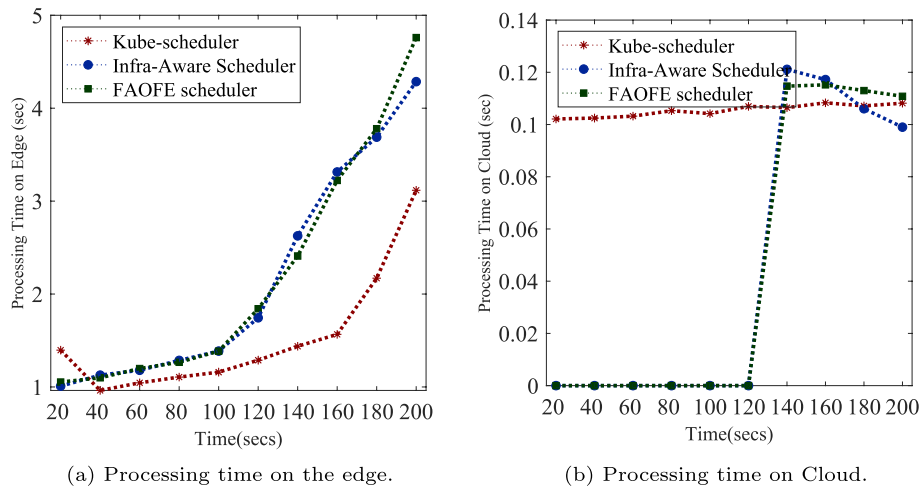


Fig. 14 Processing time on Cloud-Edge scenario for the scheduling of CAs

request the images listed in Table 4, according to the OS architecture of the worker nodes. Additionally, we set the bandwidth of the Registry Node to 10 Gbps to avoid bottlenecks in the simulation.

We generate a random number of edge AI tasks, ranging from 5 to 30 (uniformly distributed), for each containerized application. Additionally, for each containerized application, we deploy a random number of interdependent containers, ranging from 2 to 6 (uniformly distributed), as defined in the Argo workflows mechanism using the DAG concept. Each containerized EA application is then assigned a random scheduling time between 0 and 1500 seconds (uniformly distributed), determining when it must be scheduled within the edge topology.

After configuring the topology and applications for deployment, we create three scenarios, each with distinct types of SLAs related to the time required to fully instantiate the applications on the topology:

- Random Distribution: We set random SLAs with values ranging from 40 to 200 seconds for each containerized application.
- Normal Distribution: We set five possible SLAs (80, 120, 160, 200 seconds).
- 60-second SLA: We set a 60-second SLA for each containerized application without distinction.

We assume that a containerized application does not meet the SLA if any interdependent containers in the Argo workflows that compose this containerized EA application do not start until the expected time defined by the SLA. Additionally, we note that the functionality-aware strategy adopted on each node with more than one interdependent container deployed simultaneously will

cumulatively influence the schedule of all the new containerized applications.

In Fig. 15, we present the performance of each evaluation scheduling concept-based DAG in terms of minimizing SLA violations for each SLA scenario. Overall, SLA violation rates increase for the FAOFE scheduler in all three scenarios with distinct types of SLA. The results show that both the Robust-PAC time-critical workflow offloading and the JTSC managed to decrease SLA violation rates in all scenarios, resulting in 13.50%, 8.59%, 18.03%, and 19.14% smaller violation rates, respectively, compared to the FAOFE scheduler for the Random, Normal Distribution, and the 60 Sec SLA scenarios.

Additionally, the results show that both the Robust-PAC time-critical workflow offloading and the JTSC managed to achieve less than 28% SLA violation rate in scheduling the containerized applications that achieve the SLA. The Robust-PAC has a lower SLA violation rate, while the JTSC has a slightly higher rate at 5.48%. FAOFE presents that the percentage of supporting containerized applications in the best scenario (Normal Distribution) was only 62.24%. This is expected in the evaluation since the FAOFE does not prioritize network availability or the Argo workflow completion deadline time in scheduling.

In an IoT-based healthcare system with edge containerized services, DAG-based scheduling with Argo Workflows might prioritize monitoring tasks scheduled close to patients to ensure low-latency data processing. In addition, resource-intensive analysis tasks, such as predictive modeling, might be scheduled on cloud nodes with greater computational power. By considering these factors and using pre-scheduling, edge resource consumption algorithms, DAG-based scheduling can effectively choose the best edge nodes for containerized

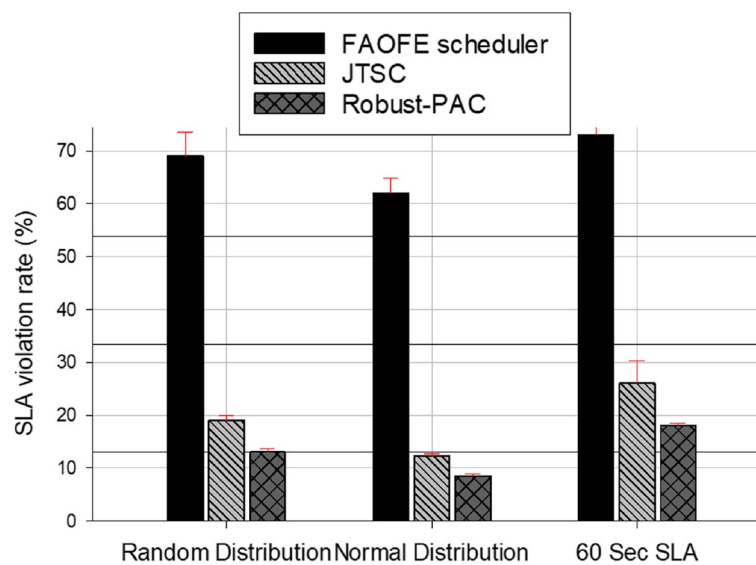


Fig. 15 SLA violation rate versus the scenarios with distinct types of SLA

applications, optimizing both performance and resource utilization across distributed environments.

Conclusion

In this study, we investigated how to offload and schedule CAs. CAs are implemented based on the microservice approach. We implemented an algorithm for orchestrating CAs by running the pre-scheduling algorithm that abstracts the Argo workflow engine on top of K8s cluster. The execution workflow configuration includes deployment workflow steps with a DAG structure. In particular, we developed a functionality-aware offloading which supports the scheduling decision by referring the edge nodes' resources. The FAOFE addressed the issue of dependencies between tasks which increase the complexity. The pre-scheduling based on Argo workflow algorithm evaluated dependencies or task order. On the issue of resource and node considerations, the complexity increased based on how many factors are being considered in node selection (e.g., CPU, memory, proximity). By applying the edge worker nodes' resource consumption decision-making method approach, the system significantly analyzed and provided the next offloading decision to edge servers. Simulation results show that FAOFE outperforms baseline scheduling schemes, and provides an efficient tradeoff between scheduling CAs and the system service rate. In a future study, we will build on this research by focusing on evaluating the proposed scheduling against extended version of Kube-Scheduler. We would investigate in future how Argo Workflows, when combined with Argo Events, can schedule workflows across different clusters. This setup allows for greater scalability and fault tolerance.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MIST) (NRF-2019R1A2C1089139).

Authors' contributions

Lionel Nkenyereye: Conceptualization, Methodology, Software, Writing- Original draft preparation. Boon Giin Lee: Data curation, Visualization, Investigation, Writing, Reviewing, and Editing. Wan-Young Chung: Supervision, Validation, Reviewing and Editing. All authors reviewed the manuscript.

Funding

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MIST) (NRF-2019R1A2C1089139).

Data availability

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

This paper does not contain any studies on human participants or animals performed by any of the authors. Informed consent was obtained from all individual participants included in the study.

Consent for publication

Informed consent for publication was obtained from all individual participants included in the study.

Competing interests

The authors declare no competing interests.

Received: 19 December 2023 Accepted: 9 February 2025

Published online: 24 February 2025

References

1. Mai ND, Lee BG, Chung WY (2021) Affective computing on machine learning-based emotion recognition using a self-made eeg device. *Sensors* 21(15). <https://doi.org/10.3390/s21155135>

2. IDC Corporate USA (n.a.) Worldwide global datasphere iot device and data forecast, 2021–2025. <https://www.idc.com/getdoc.jsp?containerId=US48087621>. Accessed 18 Jan 2023
3. Cisco (n.a.) Redefine connectivity by building a network to support the internet of things. <https://www.cisco.com/c/dam/en/us/solutions/service-provider/pdfs/a-network-to-support-iot.pdf>. Accessed 25 Aug 2022
4. Yin L, Li P, Luo J (2021) Smart contract service migration mechanism based on container in edge computing. *J Parallel Distrib Comput* 152:157–166. <https://doi.org/10.1016/j.jpdc.2021.02.023>
5. Mommessin C, Yang R, Shakhlevich NV, Sun X, Kumar S, Xiao J, Xu J (2023) Affinity-aware resource provisioning for long-running applications in shared clusters. *J Parallel Distrib Comput* 177:1–16. <https://doi.org/10.1016/j.jpdc.2023.02.011>
6. Hong CH, Varghese B (2019) Resource management in fog/edge computing: A survey on architectures, infrastructure, and algorithms. *ACM Comput Surv* 52(5). <https://doi.org/10.1145/3326066>
7. Alelaiwi A (2019) An efficient method of computation offloading in an edge cloud platform. *J Parallel Distrib Comput* 127:58–64. <https://doi.org/10.1016/j.jpdc.2019.01.003>
8. Deelman E, Vahi K, Juve G, Rynge M, Callaghan S, Maechling PJ, Mayani R, Chen W, Ferreira da Silva R, Livny M, Wenger K (2015) Pegasus, a workflow management system for science automation. *Futur Gener Comput Syst* 46:17–35. <https://doi.org/10.1016/j.future.2014.10.008>
9. Hung LH, Hu J, Meiss T, Ingersoll A, Lloyd W, Kristiyanto D, Xiong Y, Sobie E, Yeung KY (2019) Building containerized workflows using the biodepot-workflow-builder. *Cell Syst* 9(5):508–514.e3. <https://doi.org/10.1016/j.cels.2019.08.007>
10. Shan C, Xia Y, Zhan Y, Zhang J (2023) Kubeadaptor: A docking framework for workflow containerization on kubernetes. *Futur Gener Comput Syst* 148:584–599. <https://doi.org/10.1016/j.future.2023.06.022>
11. Chun B, Ha J, Oh S, Cho H, Jeong M (2019) Kubernetes enhancement for 5G NFV infrastructure. In: 2019 International Conference on Information and Communication Technology Convergence (ICTC). IEEE Operations Center, Piscataway, p 1327–1329
12. Li H, Wang Y, Huang J, Fan Y (2022) Mutation and dynamic objective-based farmland fertility algorithm for workflow scheduling in the cloud. *J Parallel Distrib Comput* 164:69–82. <https://doi.org/10.1016/j.jpdc.2022.02.005>
13. Kubernetes (2023) Kubernetes scheduler, scheduling overview and concepts. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler>. Accessed 12 Nov 2023
14. El Haj Ahmed G, Gil-Castañeira F, Costa-Montenegro E (2020) Kubcg: A dynamic kubernetes scheduler for heterogeneous clusters. *Softw Pract Exp* 51. <https://doi.org/10.1002/spe.2898>
15. Ishak H, Makhlof SA, Belalem G (2022) Kubesc-rtp: Smart scheduler for kubernetes platform on cpu-gpu heterogeneous systems. *Concurr Comput Pract Exp* 34. <https://doi.org/10.1002/cpe.7108>
16. Shan C, Wang G, Xia Y, Zhan Y, Zhang J (2021) Containerized workflow builder for kubernetes. In: 2021 IEEE 23rd Int Conf on High Performance Computing & Communications; 7th Int Conf on Data Science & Systems; 19th Int Conf on Smart City; 7th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys). pp 685–692. <https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys53884.2021.00113>
17. Santos J, Wauters T, Volckaert B, De Turck F (2019) Towards network-aware resource provisioning in kubernetes for fog computing applications. In: 2019 IEEE Conference on Network Softwareization (NetSoft). pp 351–359. <https://doi.org/10.1109/NETSOFT.2019.8806671>
18. Corodescu AA, Nikolov N, Khan AQ, Soyulu A, Matskin M, Payberah AH, Roman D (2021) Big data workflows: Locality-aware orchestration using software containers. *Sensors* 21(24)
19. Rausch T, Rashed A, Dustdar S (2020) Optimized container scheduling for data-intensive serverless edge computing. *Futur Gener Comput Syst* 114. <https://doi.org/10.1016/j.future.2020.07.017>
20. Wang L, Weng Q, Wang W, Chen C, Li B (2020) Metis: Learning to schedule long-running applications in shared container clusters at scale. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. pp 1–17. <https://doi.org/10.1109/SC41405.2020.00072>
21. Garefalakis P, Karanasos K, Pietzuch P, Suresh A, Rao S (2018) Medea: Scheduling of long running applications in shared production clusters. In: Proceedings of the Thirteenth EuroSys Conference. EuroSys '18. Association for Computing Machinery, New York. <https://doi.org/10.1145/3190508.3190549>
22. Hu S, Shi W, Li G (2023) Cec: A containerized edge computing framework for dynamic resource provisioning. *IEEE Trans Mob Comput* 22(7):3840–3854. <https://doi.org/10.1109/TMC.2022.3147800>
23. Fu S, Mittal R, Zhang L, Ratnasamy S (2020) Fast and efficient container startup at the edge via dependency scheduling. In: 3rd USENIX Workshop on Hot Topics in Edge Computing (HotEdge 20). USENIX Association. <https://www.usenix.org/conference/hotedge20/presentation/fu>. Accessed 12 Nov 2023
24. Kim Y, Park S, Shahkarami S, Sankaran R, Ferrier N, Beckman P (2022) Goal-driven scheduling model in edge computing for smart city applications. *J Parallel Distrib Comput* 167:97–108. <https://doi.org/10.1016/j.jpdc.2022.04.024>
25. Chima Ogbuachi M, Gore C, Reale A, Suskovic P, Kovács B (2019) Context-aware k8s scheduler for real time distributed 5g edge computing applications. In: 2019 International Conference on Software, Telecommunications and Computer Networks (SoftCOM). pp 1–6. <https://doi.org/10.23919/SOFTCOM.2019.8903766>
26. Kim SH, Kim T (2023) Local scheduling in kubeedge-based edge computing environment. *Sensors* 23:1522. <https://doi.org/10.3390/s23031522>
27. Dias Knob LA, Kayser CH, Ferreto T (2021) Improving container deployment in edge computing using the infrastructure aware scheduling algorithm. In: 2021 IEEE Symposium on Computers and Communications (ISCC). pp 1–6. <https://doi.org/10.1109/ISCC53001.2021.9631490>
28. Joao V, Joao P, Ricardo V (n.a.) Geolocate: A geolocation-aware scheduling system for edge computing. <https://mpvilaca.github.io/assets/pdf/VPV21.pdf>. Accessed 15 Apr 2023
29. Jiang Z, Ling N, Huang X, Shi S, Wu C, Zhao X, Yan Z, Xing G (2023) Coedge: A cooperative edge system for distributed real-time deep learning tasks. In: Proceedings of the 22nd International Conference on Information Processing in Sensor Networks. IPSN '23. Association for Computing Machinery, New York, pp 53–66. <https://doi.org/10.1145/3583120.3586955>
30. Kum S, Oh S, Yeom J, Moon J (2022) Optimization of edge resources for deep learning application with batch and model management. *Sensors* 22(17). <https://doi.org/10.3390/s22176717>
31. Tang Q, Xie R, Yu FR, Chen T, Zhang R, Huang T, Liu Y (2022) Distributed task scheduling in serverless edge computing networks for the internet of things: A learning approach. *IEEE Internet Things J* 9(20):19634–19648. <https://doi.org/10.1109/JIOT.2022.3167417>
32. Li F, Tan WJ, Cai W (2022) A wholistic optimization of containerized workflow scheduling and deployment in the cloud-edge environment. *Simul Model Pract Theory* 118:102521. <https://doi.org/10.1016/j.simpat.2022.102521>
33. k3s Leighweiht (n.a.) Lightweight kubernetes. <https://k3s.io/>. Accessed 5 Jun 2022
34. Zuk P, Rządca K (2022) Reducing response latency of composite functions-as-a-service through scheduling. *J Parallel Distrib Comput* 167:18–30. <https://doi.org/10.1016/j.jpdc.2022.04.011>
35. Gardener Universal (n.a.) Gardener universal kubernetes at scale. <https://github.com/gardener>. Accessed 26 May 2022
36. Rancher Terraform (n.a.) Rancher/terraform-controller. <https://github.com/rancher/terraform-controller>. Accessed 26 May 2022
37. Tailscale: A secure network that just works. <https://tailscale.com/>. Accessed 12 Nov 2023
38. Hub D (2022) Docker image for mobile Health pre-trained models. <https://hub.docker.com/repositories/nkenye1982>. Accessed 12 Nov 2023
39. Grafana Labs (2019) Operational dashboards for your data here, there, or anywhere. <https://grafana.com/>. Accessed 12 Nov 2023
40. Nkenyereye L (2022) Ai inference container implementation. https://github.com/nkenyelio/AI_inference_container. Accessed 12 Nov 2023
41. datasciencecom-mhealth (2019) A tutorial for datascience for classifying human activity from body motion and vital signs recordings. <https://github.com/bhimmetoglu/datasciencecom-mhealth>. Accessed 12 Nov 2023
42. CLOUDS Laboratory (n.a.) Containercloudsim: An environment for modeling and simulation of containers in cloud data center. <http://www.cloudbus.org/cloudsim/container.html>. Accessed 24 Aug 2022
43. Saleh N, Mashaly M (2019) A dynamic simulation environment for container-based cloud data centers using containercloudsim. In: 2019

- Ninth International Conference on Intelligent Computing and Information Systems (ICICIS), pp 332–336. <https://doi.org/10.1109/ICICIS46948.2019.9014697>
44. Ru J, Keung J (2013) An empirical investigation on the simulation of priority and shortest-job-first scheduling for cloud-based software systems. In: 2013 22nd Australian Software Engineering Conference. pp 78–87. <https://doi.org/10.1109/ASWEC.2013.19>
 45. Kishor A, Chakarbarty C (2021) Task offloading in fog computing for using smart ant colony optimization. *Wirel Pers Commun* 127:1–22. <https://doi.org/10.1007/s11277-021-08714-7>
 46. Ogbuachi M, Reale A, Suskovics P, Kovacs B (2020) Context-aware kubernetes scheduler for edge-native applications on 5g. *J Commun Softw Syst* 16:85. <https://doi.org/10.24138/jcomss.v16i1.1027>
 47. Zhang J, Zhou X, Ge T, Wang X, Hwang T (2021) Joint task scheduling and containerizing for efficient edge computing. *IEEE Trans Parallel Distrib Syst* 32(8):2086–2100. <https://doi.org/10.1109/TPDS.2021.3059447>
 48. Liu H, Xin R, Chen P, Gao H, Grosso P, Zhao Z (2023) Robust-pac time-critical workflow offloading in edge-to-cloud continuum among heterogeneous resources. *J Cloud Comput* 12. <https://doi.org/10.1186/s13677-023-00434-6>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.